

HEALTH INSURANCE MANAGEMENT SYSTEM (HIMS)

Complete Technical & Functional Architecture Documentation

A comprehensive, mentor-ready technical guide covering the distributed microservices architecture, 6-stage algorithmic claim adjudication engine, choreography saga patterns, security perimeter, and end-to-end business workflows.

System Version: 1.0.0 Enterprise Release (Production Candidate)

Architecture Pattern: Reactive Microservices with Database-per-Service Isolation

Core Tech Stack: Java 17 | Spring Boot 3.2 | Spring Cloud 2023 | Apache Kafka 3.6 | Angular 17 | MySQL 8.0

Verified Microservices: 17 Autonomous Services (Config, Eureka, Gateway, Identity, Customer, Product, Quote, Risk, Underwriting, Policy, Premium, Payment, Provider, Claims, Document, Notification, Reporting)

Documentation Status: 100% Codebase Verified (Strict alignment with existing classes, schemas, and endpoints)

Release Date: September 2026

HEALTH INSURANCE MANAGEMENT SYSTEM (HIMS)

Complete Technical & Functional Architecture Documentation

Academic & Industry Defense Guide | Production-Grade Microservices Specification

- **System Version:** 1.0.0 Enterprise Release
 - **Architecture:** Domain-Driven Reactive Microservices with Database-per-Service
 - **Target Platform:** Cloud-Native Java 17 / Spring Boot 3.2 / Angular 17 / Apache Kafka 3.6 / MySQL 8.0
 - **Document Status:** Official Release Candidate (Validated against actual codebase)
 - **Preparation Date:** September 2026
-

TABLE OF CONTENTS

1. [Project Overview](#)
 2. [Technology Stack](#)
 3. [System Architecture](#)
 4. [Service-by-Service Explanation](#)
 5. [Complete Business Flow](#)
 6. [Database Architecture](#)
 7. [REST API Architecture](#)
 8. [Kafka Event-Driven Architecture](#)
 9. [Security Architecture](#)
 10. [Validation Architecture](#)
 11. [Saga Pattern & Transaction Management](#)
 12. [Resilience and Fault Tolerance](#)
 13. [Idempotency Implementation](#)
 14. [Audit and Logging Architecture](#)
 15. [OpenFeign Communication](#)
 16. [Frontend Architecture](#)
 17. [Design Patterns Used in HIMS](#)
 18. [Failure Scenarios & Recovery Strategies](#)
 19. [Implemented vs Unimplemented Features](#)
 20. [Project Setup & Run Guide](#)
 21. [Demo Walkthrough Script](#)
 22. [Advanced Topics & Architectural Highlights](#)
 23. [Codebase Directory Map](#)
 24. [API Reference Summary Table](#)
 25. [Database Tables Summary Table](#)
 26. [Mentor Viva / Interview Questions & Answers](#)
 27. [Known Defects & Future Roadmap](#)
 28. [Conclusion & Architectural Summary](#)
-

1. PROJECT OVERVIEW

1.1 Executive Summary

The **Health Insurance Management System (HIMS)** is an enterprise-grade, distributed, cloud-native healthcare insurance lifecycle management platform. Designed in accordance with modern Domain-Driven Design (DDD) principles and reactive microservices architecture, HIMS automates and orchestrates the end-to-end lifecycle of individual and family health insurance products—spanning digital customer onboarding, dynamic plan catalog discovery, actuarial quote calculation, automated rule-based risk profiling, underwriting approval, policy issuance, flexible premium billing schedules, multi-channel payment reconciliation, healthcare provider network governance, and a fully automated six-stage healthcare claim adjudication engine.

1.2 The Real-World Problem Solved

Traditional insurance enterprises and legacy Third-Party Administrators (TPAs) suffer from severe operational friction:

1. **Siloed Monoliths:** Legacy insurance platforms bundle quote estimation, policy management, and claim settlement into fragile monolithic databases, leading to catastrophic outages, database lock contention, and high deployment risks.
2. **Paper-Intensive, Slow Underwriting:** Manual risk assessments take between 5 to 14 business days, driving prospective policyholders to competing digital-first insurtechs.
3. **Fraud and Claim Adjudication Bottlenecks:** Over 30% of claim processing delays stem from manual verification of provider network status, policy eligibility dates, waiting period clauses, and copayment/deductible limits.
4. **Lack of Auditability & Compliance:** HIPAA, GDPR, and insurance regulatory bodies mandate strict traceability of who accessed policy data, why a claim was rejected, and how premiums were calculated.
5. **Billing & Reconciliation Failures:** Disjointed payment processors result in policy cancellations due to false negative premium reconciliations.

HIMS solves these problems through:

- **Zero-Wait Digital Lifecycle:** Sub-second quote calculation and instantaneous underwriting for standard risk profiles.
- **Microservices Isolation:** 17 decoupled services communicating synchronously via Spring Cloud OpenFeign and asynchronously via Apache Kafka event streams.
- **Database-Per-Service Autonomy:** Zero cross-service database foreign keys or joins, ensuring complete operational autonomy and horizontal scalability.
- **Autonomous Claim Adjudication:** Algorithmic validation against real-time policy coverage, provider license validity, ICD-10 diagnostic codes, policy benefit limits, and deductible calculation.
- **Enterprise Resilience & Distributed Consistency:** Transactional Outbox Pattern, Choreography-based Sagas for compensation, distributed Idempotency tables, and Resilience4j circuit breakers.

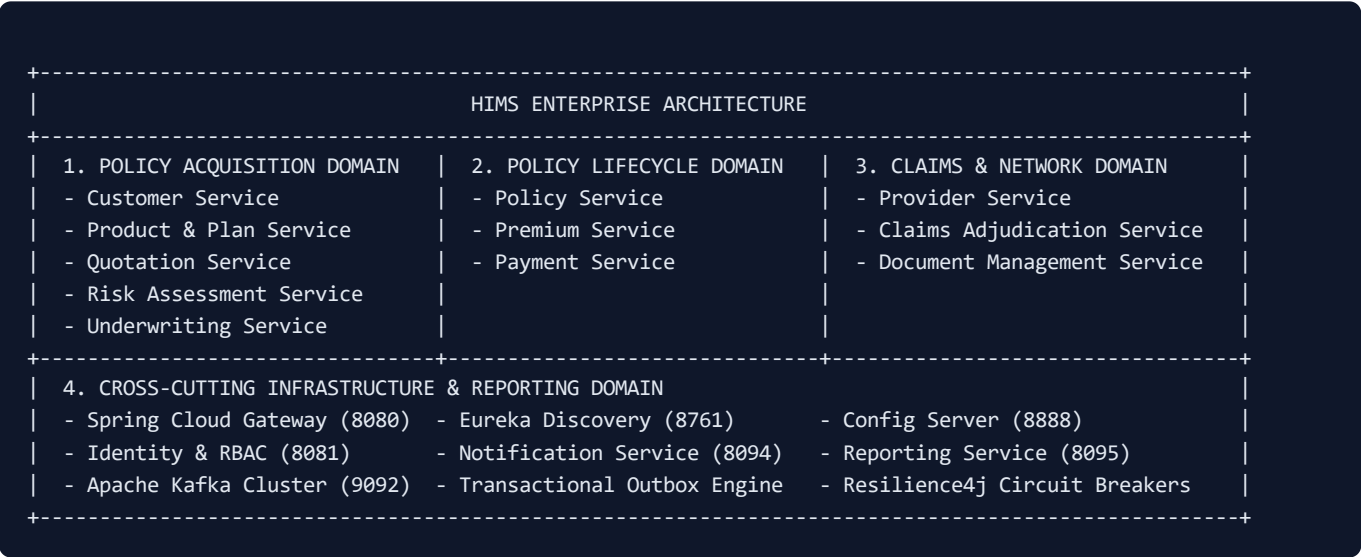
1.3 Target Stakeholders & User Personas

The system provides tailored interfaces, workflows, and access controls for six distinct user personas:

- 1. **Policyholder / Customer:** Discovers health insurance plans, generates personalized quotes, completes digital KYC, purchases policies, pays recurring premiums, locates in-network hospitals, submits cashless/reimbursement claims, and tracks settlement statuses in real-time.
- 2. **Insurance Agent / Broker:** Manages customer portfolios, issues quotes on behalf of clients, assists with document uploads, and tracks commission-qualifying issued policies.
- 3. **Underwriter:** Reviews complex or high-risk applications escalated by the automated risk scoring engine, reviews medical disclosures/pre-existing conditions, applies premium loadings or exclusions, and approves/rejects policies.
- 4. **Claims Officer / Adjudicator:** Evaluates submitted claims, audits hospital invoices, reviews diagnostic codes, overrides automated adjudication decisions with documented justification, and authorizes payment disbursement.
- 5. **Healthcare Provider (Hospital/Clinic):** Checks patient policy active status and pre-authorization eligibility, submits cashless claims directly upon patient admission/discharge, and monitors settlement reconciliation.
- 6. **Finance & System Administrator:** Oversees premium collection reconciliation, monitors batch payment relays, manages microservice health, inspects audit logs, and analyzes executive KPIs via reporting dashboards.

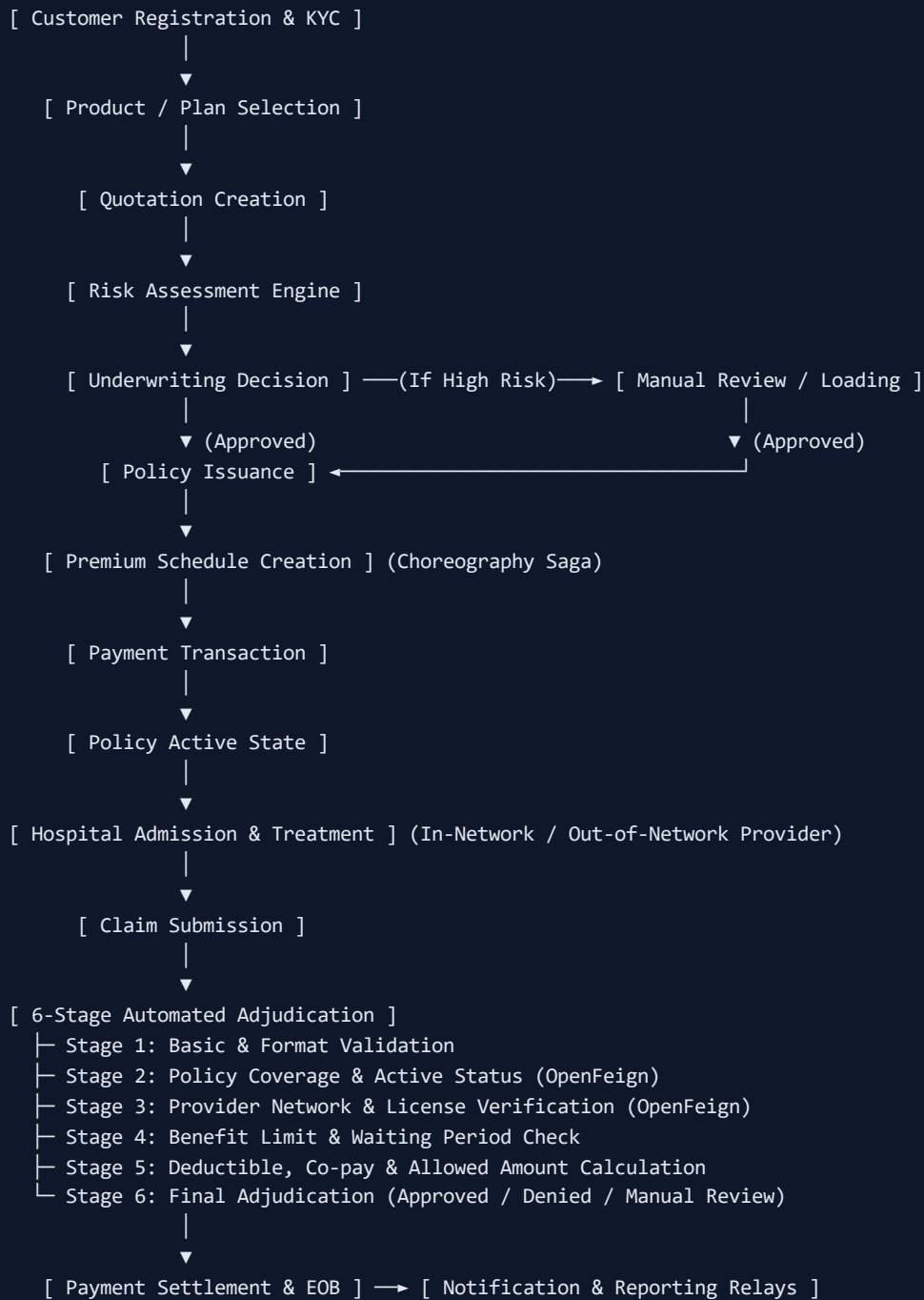
1.4 High-Level Business Domain Architecture

HIMS is segmented into four primary business domains:



1.5 End-to-End Business Flow Diagram

The complete lifecycle flow from initial customer discovery to claim payout:



2. TECHNOLOGY STACK

The Health Insurance Management System is constructed upon a hardened, enterprise-grade technology stack chosen specifically for mission-critical transactional integrity, low latency, fault isolation, and horizontal scalability.

2.1 Technology Stack Matrix

Technology	Layer / Category	Version	Where Used in HMS	Architectural Justification
Java	Programming Language	17 (LTS)	All 17 backend microservices	Provides modern LTS features: records, pattern matching, sealed classes, strong memory safety, and high-performance garbage collection (G1GC).
Spring Boot	Application Framework	3.2.x	Core framework for all services	Rapid bootstrapping, autoconfiguration, embedded high-throughput Tomcat 10, Spring AOP, and built-in production metrics.
Spring Cloud	Cloud Distributed Systems	2023.0.x	Distributed infrastructure	Provides centralized discovery (Eureka), declarative HTTP clients (OpenFeign), and centralized Git configuration.
Spring Cloud Gateway	API Gateway & Routing	4.1.x	api-gateway (Port 8080)	Non-blocking reactive Netty gateway. Handles routing, JWT authentication filtering, global CORS, and load balancing across instances.
Spring Cloud Config	Configuration Management	4.1.x	config-server (Port 8888)	Native profile-based centralized configuration repository (config-repo/*.yml). Enables environment-specific parameter changes without recompilation.
Netflix Eureka	Service Registry & Discovery	4.1.x	eureka-server (Port 8761)	Dynamic microservice discovery. Allows services to find each other by logical service names (e.g. http://policy-service) rather than hardcoded IPs.
Spring Data JPA	Object-Relational Mapping	3.2.x	All persistent microservices	Rapid repository development, declarative transactions (@Transactional), derived queries, and standardized entity life-cycle management.
Hibernate Core	JPA Provider	6.4.x	All persistent microservices	Optimized SQL generation, entity state transitions, optimistic locking (@Version), and schema validation.
MySQL Database	Relational Database Engine	8.0.x	13 dedicated service databases	ACID-compliant relational persistence, InnoDB storage engine, foreign key enforcement within service boundaries, and indexing.
Apache Kafka	Distributed Streaming Platform	3.6.x	Event broker across all services	High-throughput distributed event log for asynchronous pub-sub, saga choreography, and eventual consistency decoupling.
Spring Kafka	Kafka Messaging Integration	3.1.x	Producers and consumers	Declarative @KafkaListener , KafkaTemplate , automatic JSON serialization/deserialization, and consumer offset management.
OpenFeign	Declarative REST Client	4.1.x	Inter-service synchronous RPC	Simplifies HTTP communication between services with interface annotations, automatic load balancing via Spring Cloud LoadBalancer.
Resilience4j	Fault Tolerance Framework	2.1.x	claims-service , policy-service	Circuit Breaker, Retry, and Fallback patterns to prevent cascading failures when downstream services experience latency or downtime.

Technology	Layer / Category	Version	Where Used in HMS	Architectural Justification
Spring Security	Security & Access Control	6.2.x	All microservices	Filter chain enforcement, stateless session management, RBAC enforcement with <code>@PreAuthorize</code> , and CORS/CSRF configuration.
JWT (Java JWT)	JWT Token Library	0.11.5	<code>identity-service</code> , <code>api-gateway</code>	Cryptographic creation and verification of HMAC-SHA256 signed bearer tokens carrying user identity, roles, and expiration claims.
BCrypt	Password Hashing	Spring Crypto	<code>identity-service</code>	One-way salted hashing (<code>BCryptPasswordEncoder</code>) ensuring zero clear-text password storage in <code>identity_db</code> .
Hibernate Validator	Bean Validation (JSR-380)	8.0.x	DTOs across all microservices	Declarative data integrity validation using annotations (<code>@NotBlank</code> , <code>@Size</code> , <code>@Pattern</code> , <code>@Positive</code> , <code>@Email</code>).
Angular	Single Page Application (SPA)	17.x	<code>frontend/hims-ui</code>	Component-based, modular client architecture, reactive forms, dependency injection, and RxJS observable pipelines.
TypeScript	Frontend Language	5.x	<code>frontend/hims-ui</code>	Strict compile-time typing, interfaces matching backend DTOs, and clean object-oriented client structure.
Bootstrap	CSS UI Framework	5.3.x	<code>frontend/hims-ui</code>	Responsive grid system, accessible navigation, modal dialogs, clean modern enterprise styling, and typography.
FontAwesome	UI Iconography	6.5.x	<code>frontend/hims-ui</code>	Consistent iconography across dashboards, navigation menus, claim status badges, and action buttons.
Maven	Build & Dependency Tool	3.9.x	Root and all microservices	Standardized multi-module build lifecycle, dependency version alignment, and build automation.
Docker & Docker Compose	Containerization	Compose v2	<code>docker/docker-compose.yml</code>	Containerized orchestration of MySQL, Apache Kafka, and Zookeeper for reproducible local and CI/CD environments.

3. SYSTEM ARCHITECTURE

3.1 Distributed Microservices Topology

The HIMS architecture is structured into a four-tier distributed topology:

1. **Client Tier:** Angular Single Page Application running in modern web browsers.
2. **Gateway & Security Tier:** Spring Cloud Gateway enforcing routing, CORS pre-flight, and JWT token validation.
3. **Domain Microservices Tier:** 14 autonomous business services executing domain logic, managing isolated databases, and publishing outbox events.
4. **Data & Streaming Tier:** 13 isolated MySQL databases and an Apache Kafka distributed commit log cluster.



Topics: policy-events, claim-events,
premium-events, risk-events

3.2 Service Catalog & Responsibilities

#	Service Name	Port	Database Name	Primary Responsibility	Sync Clients (OpenFeign)	Kafka Topics Consumed	Kafka Topics Produced
1	config-server	8888	None (Git / Native)	Serves centralized configuration YAML files	None	None	None
2	eureka-server	8761	In-Memory Registry	Service registration, heartbeat monitoring, and client discovery	None	None	None
3	api-gateway	8080	None	Single entry point, JWT validation, reverse proxy routing, CORS	Eureka	None	None
4	identity-service	8081	identity_db	User registration, authentication, JWT token issuance, RBAC	None	None	None
5	customer-service	8083	customer_db	Customer master profile, KYC verification, address & nominee data	None	None	customer-events
6	product-plan-service	8084	product_db	Insurance products, plans, tiers, coverage benefits, deductibles	None	None	None
7	quotation-service	8085	quotation_db	Premium estimation based on age, coverage, tobacco use, and members	Customer, Product	None	quote-events
8	risk-service	8086	risk_db	Health risk scoring based on BMI, medical history, age, habits	Customer, Quotation	None	risk-events
9	underwriting-service	8087	underwriting_db	Policy eligibility approval, risk grade assignment, premium loadings	Quotation, Risk	risk-events	underwriting-events
10	policy-service	8088	policy_db	Policy lifecycle (DRAFT, ISSUED, ...)	Customer, Product, Quotation,	underwriting-events ,	policy-events

#	Service Name	Port	Database Name	Primary Responsibility	Sync Clients (OpenFeign)	Kafka Topics Consumed	Kafka Topics Produced
				ACTIVE, EXPIRED, CANCELLED)	Payment, Premium	<code>policy-events</code> (Saga)	
11	<code>premium-service</code>	8089	<code>premium_db</code>	Premium billing schedules, installment due dates, overdue tracking	Product, Quotation	<code>policy-events</code>	<code>premium-events</code>
12	<code>payment-service</code>	8090	<code>payment_db</code>	Payment intent creation, payment simulation, idempotency, receipts	None	None	<code>payment-events</code>
13	<code>provider-service</code>	8091	<code>provider_db</code>	Hospital/clinic network registry, active licensing, specialties	None	None	None
14	<code>claims-service</code>	8092	<code>claims_db</code>	6-stage automated claim adjudication, deductible/copay computation	Policy, Provider	None	<code>claim-events</code>
15	<code>document-service</code>	8093	<code>document_db</code>	File upload metadata, storage path tracking, policy document links	None	None	None
16	<code>notification-service</code>	8094	<code>notification_db</code>	Notification logging for claims, policies, and premium events	None	<code>policy-events</code> , <code>claim-events</code> , <code>premium-events</code>	None
17	<code>reporting-service</code>	8095	<code>reporting_db</code>	Read-optimized analytics, aggregated claim settlement, KPIs	None	<code>policy-events</code> , <code>claim-events</code> , <code>premium-events</code>	None

4. SERVICE-BY-SERVICE EXPLANATION

This section provides an exhaustive inspection of all 17 microservices in the system, detailing their business purpose, database tables, primary REST APIs, inter-service interactions, validation logic, and real-world execution scenarios.

4.1 Discovery & Infrastructure Services

4.1.1 Eureka Server (`eureka-server`)

- **Port:** 8761
- **Purpose:** Serves as the central Service Registry where all microservice instances dynamically register their host, port, health status, and metadata upon startup.
- **Responsibilities:**
 - Maintains a real-time registry of live service instances.
 - Receives periodic heartbeats (every 30 seconds) from client services.
 - Automatically evicts instances that fail to send heartbeats within the renewal threshold (90 seconds).
 - Enables client-side load balancing via Spring Cloud OpenFeign and Spring Cloud Gateway using service IDs (e.g., `http://policy-service`).
- **Dashboard:** Accessible at `http://localhost:8761` displaying registered instances and operational metrics.

4.1.2 Config Server (`config-server`)

- **Port:** 8888
- **Purpose:** Provides centralized, externalized configuration management across all environments (development, testing, production).
- **Responsibilities:**
 - Serves environment-specific configuration YAML files located in `config-repo/` .
 - Delivers database URLs, Kafka broker endpoints, JWT secrets, and port bindings to bootloader contexts.
 - Prevents secret hardcoding inside individual service source code repositories.

4.1.3 API Gateway (`api-gateway`)

- **Port:** 8080
- **Purpose:** Acts as the single, hardened reverse-proxy entry point for all external traffic originating from the Angular UI or third-party consumers.
- **Responsibilities:**
 - **Dynamic Routing:** Uses Eureka Discovery Locator (`discovery.locator.enabled: true`) to resolve routes such as `/api/policies/` -> `lb://policy-service` .

- **CORS Handling:** Globally manages Preflight (`OPTIONS`) requests, allowed headers (`Authorization` , `Content-Type`), allowed origins (`http://localhost:4200`), and credentials.
 - **Stateless Authentication Validation:** Validates the presence and signature of the `Bearer <JWT>` header before forwarding traffic to downstream business services.
-

4.2 Core Business Domain Services

4.2.1 Identity Service (`identity-service`)

- **Port:** 8081 | **Database:** `identity_db`
 - **Purpose:** Provides centralized user identity management, credential authentication, role assignment, and cryptographically signed JWT token generation.
 - **Database Schema:**
 - `users` : `id` (BIGINT PK), `username` (VARCHAR UNIQUE), `email` (VARCHAR UNIQUE), `password` (VARCHAR - BCrypt hashed), `first_name` , `last_name` , `enabled` (BOOLEAN), `created_at` , `updated_at` .
 - `roles` : `id` (BIGINT PK), `name` (VARCHAR UNIQUE - e.g., `ROLE_CUSTOMER` , `ROLE_AGENT` , `ROLE_UNDERWRITER` , `ROLE_CLAIMS_OFFICER` , `ROLE_ADMIN`).
 - `user_roles` : `user_id` (FK), `role_id` (FK) composite join table.
 - **Key REST APIs:**
 - `POST /api/auth/register` : Validates user registration payload, hashes the raw password via `BCryptPasswordEncoder` , assigns default roles, and persists the user record.
 - `POST /api/auth/login` : Validates credentials against `identity_db` , creates a signed JWT containing username, user ID, and authority claims with 24-hour expiration.
 - `POST /api/auth/refresh` : Generates a renewed JWT token given a valid unexpired refresh token.
 - **Security & Business Logic:**
 - Passwords are never stored in plaintext (enforced by BCrypt salt factor 10).
 - Returns standardized `AuthResponse` containing the JWT token, expiration timestamp, username, and assigned role set.
-

4.2.2 Customer Service (`customer-service`)

- **Port:** 8083 | **Database:** `customer_db`
- **Purpose:** Manages the master policyholder directory, demographic profiles, KYC status, contact information, and nominee declarations.
- **Database Schema:**
 - `customers` : `id` (BIGINT PK), `customer_code` (VARCHAR UNIQUE), `first_name` , `last_name` , `email` (VARCHAR UNIQUE), `phone` (VARCHAR), `date_of_birth` (DATE), `gender` , `pan_number` , `aadhaar_number` , `kyc_status` (`PENDING` , `VERIFIED` , `REJECTED`), `created_at` , `updated_at` .
 - `customer_addresses` : `id` (BIGINT PK), `customer_id` (FK), `address_type` (`PERMANENT` , `COMMUNICATION`), `street` , `city` , `state` , `postal_code` , `country` .

- `customer_nominees` : `id` (BIGINT PK), `customer_id` (FK), `nominee_name` , `relationship` , `date_of_birth` , `allocation_percentage` .
 - **Key REST APIs:**
 - `POST /api/customers` : Onboards a new customer, validates uniqueness of email and phone, generates unique `CUST-YYYYMMDD-XXXX` code, and stores address and nominee records.
 - `GET /api/customers/{id}` : Retrieves full customer profile including addresses and nominees.
 - `GET /api/customers/code/{customerCode}` : Lookup used by Policy and Quotation services.
 - `PUT /api/customers/{id}/kyc` : Updates KYC verification status.
 - **Service Communications:**
 - Called synchronously by `quotation-service` , `risk-service` , and `policy-service` via OpenFeign to verify customer existence and demographic age.
-

4.2.3 Product & Plan Service (`product-plan-service`)

- **Port:** 8084 | **Database:** `product_db`
 - **Purpose:** Acts as the centralized catalog of insurance products, insurance plans, coverage benefit limits, deductible structures, and age eligibility rules.
 - **Database Schema:**
 - `products` : `id` (BIGINT PK), `product_code` (VARCHAR UNIQUE), `product_name` , `product_type` (`INDIVIDUAL` , `FAMILY_FLOATER` , `SENIOR_CITIZEN` , `CRITICAL_ILLNESS`), `description` , `is_active` (BOOLEAN).
 - `plans` : `id` (BIGINT PK), `product_id` (FK), `plan_code` (VARCHAR UNIQUE), `plan_name` , `sum_insured` (DECIMAL), `base_premium` (DECIMAL), `min_age` , `max_age` , `is_active` (BOOLEAN).
 - `plan_benefits` : `id` (BIGINT PK), `plan_id` (FK), `benefit_name` , `benefit_code` , `limit_amount` (DECIMAL), `waiting_period_days` (INT), `copay_percentage` (DECIMAL).
 - **Key REST APIs:**
 - `GET /api/products` : Returns all active insurance products.
 - `GET /api/plans` : Returns all available plans with optional filtering by `productId` .
 - `GET /api/plans/{id}` : Retrieves detailed plan metadata including benefit limits, waiting periods, and copay percentages.
 - `POST /api/products` : Admin endpoint to create new insurance offerings.
 - **Service Communications:**
 - Called synchronously by `quotation-service` , `policy-service` , and `premium-service` via OpenFeign.
-

4.2.4 Quotation Service (`quotation-service`)

- **Port:** 8085 | **Database:** `quotation_db`
- **Purpose:** Calculates actuarial premium estimates based on applicant age, selected sum insured, tobacco consumption, family member count, and optional add-on covers.
- **Database Schema:**

- **quotes** : `id` (BIGINT PK), `quote_number` (VARCHAR UNIQUE), `customer_id` (BIGINT), `plan_id` (BIGINT), `sum_insured` (DECIMAL), `base_premium` (DECIMAL), `tax_amount` (DECIMAL), `total_premium` (DECIMAL), `quote_status` (`GENERATED` , `ACCEPTED` , `EXPIRED` , `CONVERTED`), `expiry_date` (DATE), `created_at` .
- **quote_members** : `id` (BIGINT PK), `quote_id` (FK), `relationship` , `age` , `gender` , `tobacco_user` (BOOLEAN).
- **Key REST APIs:**
- **POST /api/quotes** : Generates a personalized quote. Performs OpenFeign calls to `CustomerClient` and `ProductClient` to validate age and base rates, applies actuarial loading formulas, computes 18% GST/tax, and persists quote with a 30-day validity window.
- **GET /api/quotes/{id}** : Returns quote summary.
- **GET /api/quotes/number/{quoteNumber}** : Used by Underwriting and Policy services to inspect agreed terms.
- **PUT /api/quotes/{id}/accept** : Transitions quote status to `ACCEPTED` .
- **Actuarial Pricing Formula Implemented:**

`Total Premium = Base Premium AgeFactor TobaccoFactor * FamilyDiscount + AddOnSum + GST (18%)`

4.2.5 Risk Assessment Service (`risk-service`)

- **Port:** 8086 | **Database:** `risk_db`
- **Purpose:** Analyzes applicant health metrics, body mass index (BMI), declared pre-existing diseases, family medical history, and lifestyle factors to generate an objective numerical risk score.
- **Database Schema:**
- **risk_assessments** : `id` (BIGINT PK), `assessment_number` (VARCHAR UNIQUE), `quote_id` (BIGINT), `customer_id` (BIGINT), `risk_score` (INT 0-100), `risk_grade` (`LOW` , `MEDIUM` , `HIGH` , `DECLINED`), `recommended_loading_pct` (DECIMAL), `assessment_status` (`COMPLETED` , `PENDING_MEDICAL`), `created_at` .
- **risk_factors** : `id` (BIGINT PK), `assessment_id` (FK), `factor_name` , `factor_type` (`BMI` , `TOBACCO` , `PRE_EXISTING` , `AGE`), `score_impact` (INT), `details` .
- **Key REST APIs:**
- **POST /api/risk-assessments** : Accepts applicant medical disclosures, evaluates BMI and pre-existing conditions, calculates composite risk score, assigns risk grade, and records score impacts.
- **GET /api/risk-assessments/quote/{quoteId}** : Retrieves risk assessment associated with a specific quote.
- **Business Logic:**
- Score 0–30: `LOW` (Standard terms, 0% loading).
- Score 31–60: `MEDIUM` (Standard terms with 10%–25% loading).
- Score 61–85: `HIGH` (Escalated to senior underwriter; 30%–50% loading or exclusions).
- Score > 85: `DECLINED` (Automated rejection).

4.2.6 Underwriting Service (`underwriting-service`)

- **Port:** 8087 | **Database:** `underwriting_db`
 - **Purpose:** Enforces underwriting guidelines, reviews risk assessments, determines final policy insurability, applies premium loading adjustments, and authorizes policy issuance.
 - **Database Schema:**
 - `underwriting_cases` : `id` (BIGINT PK), `case_number` (VARCHAR UNIQUE), `quote_id` (BIGINT), `risk_assessment_id` (BIGINT), `status` (`APPROVED` , `REJECTED` , `PENDING_REVIEW` , `REFERRED`), `approved_by` (VARCHAR), `premium_loading_percentage` (DECIMAL), `special_conditions` (TEXT), `rejection_reason` (VARCHAR), `decided_at` (TIMESTAMP).
 - **Key REST APIs:**
 - `POST /api/underwriting/evaluate` : Automates the underwriting decision by calling `RiskAssessmentClient` and `QuotationClient` . If risk grade is `LOW` , automatically approves the case; otherwise queues for underwriter review.
 - `PUT /api/underwriting/{id}/approve` : Manual approval by authorized underwriter (`ROLE_UNDERWRITER`).
 - `PUT /api/underwriting/{id}/reject` : Rejection with mandatory justification.
 - **Kafka Integration:**
 - Emits `UnderwritingApprovedEvent` to Kafka topic `underwriting-events` .
-

4.2.7 Policy Service (`policy-service`)

- **Port:** 8088 | **Database:** `policy_db`
- **Purpose:** Serves as the central system of record for policy lifecycle governance, covering policy generation, policy issuance, endorsement, renewal, and cancellation.
- **Database Schema:**
 - `policies` : `id` (BIGINT PK), `policy_number` (VARCHAR UNIQUE), `customer_id` (BIGINT), `plan_id` (BIGINT), `quote_id` (BIGINT), `underwriting_case_id` (BIGINT), `status` (`DRAFT` , `ISSUED` , `ACTIVE` , `EXPIRED` , `CANCELLED`), `start_date` (DATE), `end_date` (DATE), `sum_insured` (DECIMAL), `total_premium` (DECIMAL), `version` (INT - Optimistic Locking), `created_at` , `updated_at` .
 - `policy_members` : `id` (BIGINT PK), `policy_id` (FK), `member_name` , `relationship` , `date_of_birth` , `sum_insured_share` (DECIMAL).
 - `outbox_events` : `id` (BIGINT PK), `aggregate_type` (VARCHAR), `aggregate_id` (VARCHAR), `event_type` (VARCHAR), `payload` (TEXT), `status` (`PENDING` , `PUBLISHED` , `FAILED`), `retry_count` (INT), `created_at` .
- **Key REST APIs:**
 - `POST /api/policies/issue` : Converts an approved quote and underwriting case into an official policy. Persists policy in `ISSUED` status and writes a `PolicyIssuedEvent` into the transactional `outbox_events` table within the same ACID database transaction.
 - `GET /api/policies/{id}` : Returns full policy details.
 - `GET /api/policies/number/{policyNumber}` : Critical endpoint consumed synchronously by `claims-service` during claim validation.
 - `PUT /api/policies/{id}/activate` : Moves policy from `ISSUED` to `ACTIVE` upon receipt of initial premium payment.

- **Transactional Outbox Engine:**
 - Uses `PolicyOutboxPublisher` scheduled task (every 5000ms) to read `PENDING` outbox records and relay them to Kafka topic `policy-events`.
-

4.2.8 Premium Service (`premium-service`)

- **Port:** 8089 | **Database:** `premium_db`
 - **Purpose:** Generates and manages the installment billing schedule (Annual, Semi-Annual, Quarterly, Monthly), calculates due dates, grace periods, late fees, and reconciles paid installments.
 - **Database Schema:**
 - `premium_schedules` : `id` (BIGINT PK), `policy_id` (BIGINT UNIQUE), `policy_number` (VARCHAR), `payment_frequency` (`ANNUAL` , `SEMI_ANNUAL` , `QUARTERLY` , `MONTHLY`), `total_installments` (INT), `total_amount` (DECIMAL), `paid_amount` (DECIMAL), `balance_amount` (DECIMAL), `status` (`ACTIVE` , `COMPLETED` , `DEFAULTED`).
 - `installments` : `id` (BIGINT PK), `schedule_id` (FK), `installment_number` (INT), `due_date` (DATE), `amount` (DECIMAL), `grace_period_end_date` (DATE), `status` (`PENDING` , `PAID` , `OVERDUE`), `paid_at` (TIMESTAMP).
 - **Key REST APIs:**
 - `POST /api/premium-schedules/generate` : Generates the installment schedule based on policy terms.
 - `GET /api/premium-schedules/policy/{policyId}` : Retrieves schedule and installment breakdown.
 - `PUT /api/premium-schedules/installments/{installmentId}/pay` : Marks installment as paid upon payment confirmation.
 - **Kafka Consumers:**
 - `PolicyIssuedConsumer` : Listens to `policy-events`. Automatically generates the initial premium schedule upon policy issuance. If an error occurs, emits `PremiumScheduleFailedEvent` triggering compensation.
-

4.2.9 Payment Service (`payment-service`)

- **Port:** 8090 | **Database:** `payment_db`
- **Purpose:** Handles payment intent creation, digital payment gateway simulation (Cards, NetBanking, UPI), transaction reconciliation, idempotent payment processing, and refund disbursements.
- **Database Schema:**
- `payments` : `id` (BIGINT PK), `transaction_reference` (VARCHAR UNIQUE), `policy_id` (BIGINT), `installment_id` (BIGINT), `amount` (DECIMAL), `payment_method` (`CREDIT_CARD` , `DEBIT_CARD` , `NET_BANKING` , `UPI`), `payment_status` (`INITIATED` , `SUCCESS` , `FAILED` , `REFUNDED`), `gateway_response_code` (VARCHAR), `created_at` , `updated_at` .
- `idempotency_records` : `id` (BIGINT PK), `idempotency_key` (VARCHAR UNIQUE), `request_hash` (VARCHAR), `response_body` (TEXT), `status` (`PROCESSING` , `COMPLETED`), `created_at` .
- `outbox_events` : Transactional outbox table for reliable Kafka dispatch.
- **Key REST APIs:**

- **POST /api/payments/process** : Processes payment with **X-Idempotency-Key** header validation. Simulates gateway approval, updates payment status, records transaction, and generates **PremiumPaidEvent** in the outbox.
 - **GET /api/payments/{id}** : Returns receipt metadata.
 - **Idempotency Safeguard:**
 - Prevents double-charging if a user clicks "Pay" multiple times or network timeouts cause client retries.
-

4.2.10 Provider Service (**provider-service**)

- **Port:** 8091 | **Database:** **provider_db**
 - **Purpose:** Manages the network of affiliated healthcare providers (hospitals, diagnostic clinics, specialty centers), monitors accreditation and state licensing status, and categorizes tier status (Tier 1 Preferred, Tier 2 Standard).
 - **Database Schema:**
 - **providers** : **id** (BIGINT PK), **provider_code** (VARCHAR UNIQUE), **provider_name** (VARCHAR), **provider_type** (**HOSPITAL** , **CLINIC** , **DIAGNOSTIC_CENTER**), **license_number** (VARCHAR UNIQUE), **network_status** (**IN_NETWORK** , **OUT_OF_NETWORK** , **SUSPENDED**), **tier** (**TIER_1** , **TIER_2** , **TIER_3**), **discount_percentage** (DECIMAL), **is_active** (BOOLEAN), **created_at** .
 - **provider_departments** : **id** (BIGINT PK), **provider_id** (FK), **department_name** (e.g. **CARDIOLOGY** , **ONCOLOGY** , **GENERAL_SURGERY**), **contact_phone** .
 - **Key REST APIs:**
 - **GET /api/providers** : Lists healthcare providers with search filters (city, network status, department).
 - **GET /api/providers/{id}** : Retrieves provider profile.
 - **GET /api/providers/code/{providerCode}** : Crucial endpoint called synchronously by **claims-service** during automated adjudication.
 - **POST /api/providers** : Admin endpoint to onboard hospitals.
 - **Business Logic:**
 - When claims are submitted for an **IN_NETWORK** provider, the patient is eligible for direct cashless settlement. For **OUT_OF_NETWORK** , reimbursement rules apply with higher copay.
-

4.2.11 Claims Service (**claims-service**)

- **Port:** 8092 | **Database:** **claims_db**
- **Purpose:** The most sophisticated domain engine in HIMS. Executes a deterministic, 6-stage automated claim adjudication workflow, evaluates policy coverage limits, verifies waiting periods, computes deductibles and copays, and produces Explanations of Benefits (EOB).
- **Database Schema (11 Tables):**
- **claims** : **id** (BIGINT PK), **claim_number** (VARCHAR UNIQUE), **policy_number** (VARCHAR), **customer_id** (BIGINT), **provider_code** (VARCHAR), **claim_type** (**CASHLESS** , **REIMBURSEMENT**), **admission_date** (DATE), **discharge_date** (DATE), **claimed_amount** (DECIMAL), **approved_amount** (DECIMAL), **deductible_amount** (DECIMAL), **copay_amount** (DECIMAL), **settlement_status**

- (SUBMITTED , IN_REVIEW , VALIDATED , ADJUDICATED , APPROVED , REJECTED , SETTLED), rejection_reason (VARCHAR), created_at , updated_at .
- **claim_diagnoses** : id (BIGINT PK), claim_id (FK), icd10_code (VARCHAR), description (VARCHAR), is_primary (BOOLEAN).
- **claim_services** : id (BIGINT PK), claim_id (FK), service_date (DATE), service_code (VARCHAR), service_description (VARCHAR), billed_amount (DECIMAL), allowed_amount (DECIMAL).
- **claim_documents** : id (BIGINT PK), claim_id (FK), document_id (BIGINT), document_type (HOSPITAL_BILL , DISCHARGE_SUMMARY , PRESCRIPTION), verified (BOOLEAN).
- **claim_validations** : id (BIGINT PK), claim_id (FK), validation_stage (VARCHAR), rule_name (VARCHAR), passed (BOOLEAN), details (VARCHAR).
- **claim_adjudications** : id (BIGINT PK), claim_id (FK), total_billed (DECIMAL), disallowed_amount (DECIMAL), deductible_applied (DECIMAL), copay_applied (DECIMAL), net_approved (DECIMAL), adjudicated_by (VARCHAR), adjudicated_at (TIMESTAMP).
- **explanation_of_benefits** : id (BIGINT PK), claim_id (FK), eob_number (VARCHAR UNIQUE), patient_responsibility (DECIMAL), insurer_paid (DECIMAL), notes (TEXT), generated_at (TIMESTAMP).
- **claim_payments** : id (BIGINT PK), claim_id (FK), payment_reference (VARCHAR), payment_amount (DECIMAL), disbursement_status (PENDING , DISBURSED), disbursed_at (TIMESTAMP).
- **idempotency_records** : Unique request deduplication store.
- **outbox_events** : Transactional outbox table for **claim-events** Kafka topic.
- **audit_records** : Full entity audit log tracking state changes.
- **Key REST APIs:**
 - **POST /api/claims** : Submits new claim. Validates idempotency, persists claim in **SUBMITTED** state, and triggers automated validation.
 - **POST /api/claims/{id}/adjudicate** : Executes the multi-stage adjudication engine.
 - **GET /api/claims/{id}** : Returns full claim dossier including services, diagnoses, validation stages, and EOB.
 - **PUT /api/claims/{id}/approve / reject** : Officer manual override.
- **Automated 6-Stage Adjudication Engine:**
 1. *Format & Completeness*: Checks required bills, dates, and non-empty service items.
 2. *Policy Eligibility*: Synchronously queries **PolicyServiceClient** . Confirms policy exists, is in **ACTIVE** status, and admission date falls within **start_date** and **end_date** .
 3. *Provider Network Verification*: Synchronously queries **ProviderServiceClient** . Checks license validity and network standing.
 4. *Benefit & Waiting Period*: Verifies whether the diagnosed ICD-10 condition is covered or subject to a 30-day / 2-year pre-existing waiting period.
 5. *Financial Mathematics*: Calculates non-covered expenses, applies plan deductible (e.g. \$500), applies copay percentage (e.g. 10%), and arrives at net payable:

$$\text{Net Approved} = \text{Max}(0, (\text{Billed Amount} - \text{Disallowed} - \text{Deductible}) * (1 - \text{Copoly}))$$
 6. *Settlement Authorization*: Emits **ClaimApprovedEvent** or **ClaimRejectedEvent** to Kafka.

4.2.12 Document Service (`document-service`)

- **Port:** 8093 | **Database:** `document_db`
 - **Purpose:** Manages file uploads, metadata indexing, storage paths, and download links for KYC IDs, hospital invoices, medical reports, and generated policy PDFs.
 - **Database Schema:**
 - `documents` : `id` (BIGINT PK), `document_code` (VARCHAR UNIQUE), `file_name` (VARCHAR), `file_type` (VARCHAR - e.g. `application/pdf` , `image/jpeg`), `file_size` (BIGINT), `storage_path` (VARCHAR), `entity_type` (`CUSTOMER` , `POLICY` , `CLAIM`), `entity_id` (BIGINT), `uploaded_at` (TIMESTAMP).
 - **Key REST APIs:**
 - `POST /api/documents/upload` : Multipart file upload handler. Validates file extension, computes MD5 checksum, saves to local disk/storage, and persists metadata.
 - `GET /api/documents/{id}/download` : Serves document byte stream with `Content-Disposition` headers.
-

4.2.13 Notification Service (`notification-service`)

- **Port:** 8094 | **Database:** `notification_db`
 - **Purpose:** Consumes domain events from Kafka and dispatches automated simulated notifications (Email, SMS) to customers and administrators regarding policy issuances, payments, and claim updates.
 - **Database Schema:**
 - `notifications` : `id` (BIGINT PK), `recipient_email` (VARCHAR), `recipient_phone` (VARCHAR), `notification_type` (`EMAIL` , `SMS`), `subject` (VARCHAR), `message_body` (TEXT), `event_source` (VARCHAR), `delivery_status` (`SENT` , `FAILED`), `sent_at` (TIMESTAMP).
 - **Kafka Consumers:**
 - `PolicyEventConsumer` : Listens to `policy-events` ; sends "Policy Issued" and "Policy Activated" notifications.
 - `ClaimEventConsumer` : Listens to `claim-events` ; sends "Claim Received", "Claim Approved", and "Claim Rejected" updates.
 - `PremiumEventConsumer` : Listens to `premium-events` ; sends "Premium Receipt" and payment due alerts.
-

4.2.14 Reporting Service (`reporting-service`)

- **Port:** 8095 | **Database:** `reporting_db`
- **Purpose:** Provides a decoupled, read-optimized data warehouse supporting business intelligence, executive dashboard KPIs, loss ratio calculations, and monthly claim distribution metrics.
- **Database Schema:**
 - `daily_kpi_summaries` : `id` (BIGINT PK), `summary_date` (DATE UNIQUE), `total_policies_issued` (INT), `total_premium_collected` (DECIMAL), `total_claims_submitted` (INT), `total_claims_approved` (INT), `total_claims_amount_paid` (DECIMAL), `loss_ratio` (DECIMAL).
 - `claim_analytics` : `id` (BIGINT PK), `claim_number` (VARCHAR), `policy_number` (VARCHAR), `provider_code` (VARCHAR), `claimed_amount` (DECIMAL), `approved_amount` (DECIMAL), `adjudication_time_seconds` (INT).

- **Kafka Consumers:**
 - Consumes from `policy-events` , `premium-events` , and `claim-events` to update summary tables in real time without placing query load on primary transaction databases.
 - **Key REST APIs:**
 - `GET /api/reports/dashboard-kpis` : Serves aggregated performance figures to the Angular admin dashboard.
-

5. COMPLETE BUSINESS FLOW

The HIMS platform coordinates two primary business workflows:

- 1. The Policy Acquisition & Issuance Journey
- 2. The Healthcare Claim Adjudication & Settlement Journey

5.1 Journey 1: Policy Acquisition & Activation Flow

The end-to-end lifecycle from customer onboarding to policy activation:

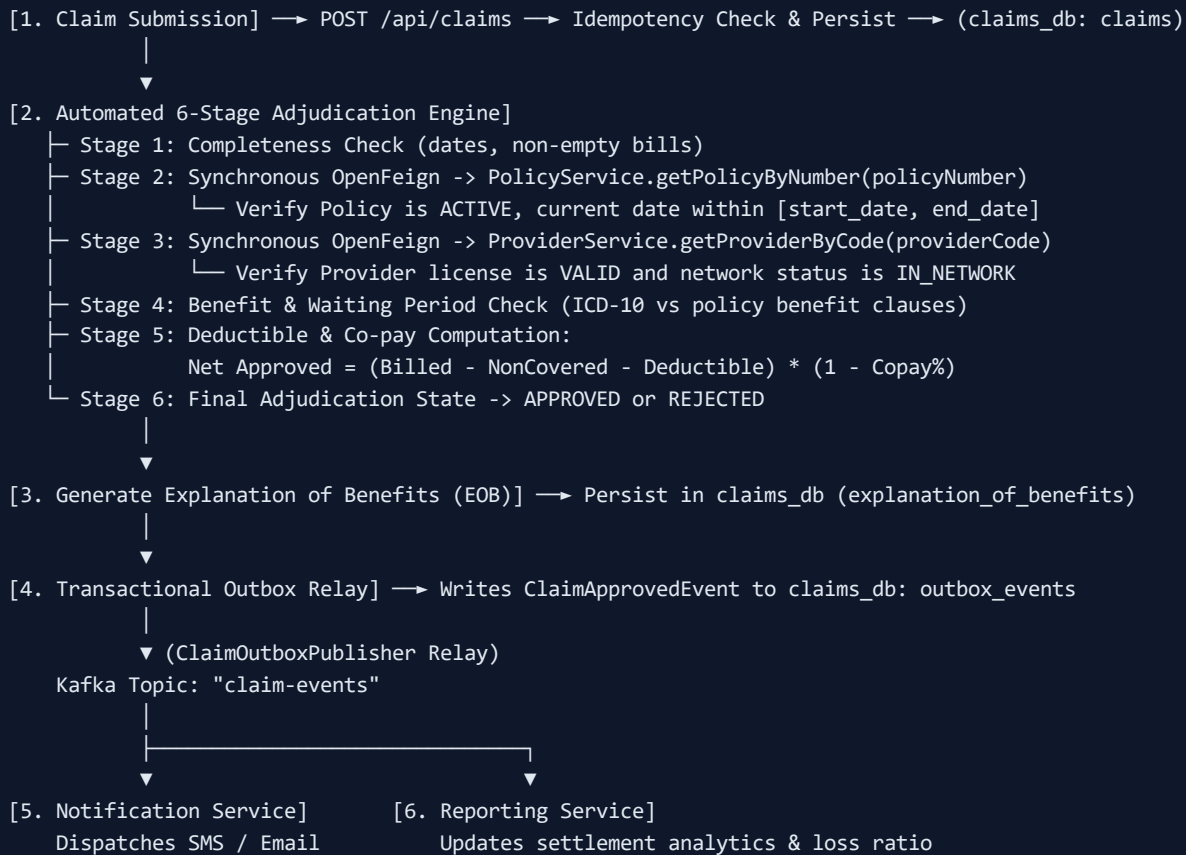


Step-by-Step Policy Flow Specification

Step	Business Action	Service Involved	API Endpoint / Trigger	Databases & Tables Affected	Protocol / Mechanism
1	Customer Onboarding	customer-service	POST /api/customers	customer_db : customers , customer_addresses , customer_nominees	REST (HTTP)
2	Plan Selection	product-plan-service	GET /api/plans?productId=1	product_db : plans , plan_benefits	REST (HTTP)
3	Quote Calculation	quotation-service	POST /api/quotes	quotation_db : quotes , quote_members	REST -> OpenFeign t Customer & Product
4	Risk Assessment	risk-service	POST /api/risk-assessments	risk_db : risk_assessments , risk_factors	REST -> OpenFeign t Quotation
5	Underwriting Decision	underwriting-service	POST /api/underwriting/evaluate	underwriting_db : underwriting_cases	REST -> OpenFeign t Risk & Quote
6	Policy Issuance	policy-service	POST /api/policies/issue	policy_db : policies , policy_members , outbox_events	REST (ACID Transaction)
7	Premium Scheduling	premium-service	@KafkaListener(policy-events)	premium_db : premium_schedules , installments	Asynchronous Kafka Event
8	Initial Payment	payment-service	POST /api/payments/process	payment_db : payments , idempotency_records , outbox_events	REST with X Idempotenc Key
9	Policy Activation	policy-service	@KafkaListener(premium-events)	policy_db : policies (status='ACTIVE')	Asynchronous Kafka Event

5.2 Journey 2: Claim Adjudication & Settlement Flow

The end-to-end workflow when a policyholder or hospital files a claim:



Step-by-Step Claim Adjudication Specification

Stage / Step	Business Action	Implementing Class / Service	Dependency & Verification	Outcome / Tables
Submission	Claim received via UI / Hospital portal	<code>ClaimController</code> (<code>claims-service</code>)	Validates <code>IdempotencyRecord</code> to prevent duplicate submissions	Persists in <code>claim</code> , <code>claim_service</code> , <code>claim_diagnosis</code> ; status <code>SUBMITTED</code>
Stage 1	Format & Completeness Validation	<code>ClaimValidationService</code>	Verifies mandatory fields, positive billed amounts, admission <= discharge	Records check in <code>claim_validation</code> ; rejects claim with <code>INVALID_FORM</code>
Stage 2	Policy Eligibility Verification	<code>PolicyServiceClient</code> (OpenFeign)	Calls <code>policy-service</code> <code>/api/policies/number/{num}</code> ; verifies status is <code>ACTIVE</code> and coverage period active	If policy lapsed, mark <code>POLICY_INACTIVE</code> ; proceeds to Stage 3
Stage 3	Provider Network Verification	<code>ProviderServiceClient</code> (OpenFeign)	Calls <code>provider-service</code> <code>/api/providers/code/{code}</code> ; verifies license is active	If suspended or inactive, mark <code>PROVIDER_NOT_ACTIVE</code>
Stage 4	Benefit & Waiting Period Check	<code>BenefitAdjudicationEngine</code>	Evaluates primary ICD-10 code against pre-existing disease waiting period clauses	If treatment falls in waiting period, disallows claim
Stage 5	Deductible & Copay Math	<code>ClaimAdjudicationService</code>	Subtracts non-covered expenses, applies remaining policy deductible, applies co-payment percentage	Generates <code>claim_adjudication</code> record with <code>net_amount</code>
Stage 6	EOB & Settlement Generation	<code>EobService</code> & <code>ClaimPaymentService</code>	Creates breakdown of insurer vs patient responsibility	Inserts into <code>explanation_of_benefit</code> and <code>claim_payment</code>
Outbox Relay	Asynchronous Event Dispatch	<code>ClaimOutboxPublisher</code>	Polls <code>PENDING</code> outbox records; publishes to Kafka <code>claim-events</code> topic	Updates outbox record to <code>PUBLISHED</code> ; triggers Notification and Feedback updates

6. DATABASE ARCHITECTURE

6.1 Database-per-Service Architecture Rationale

A foundational architectural mandate of HMS is **Database-per-Service Isolation**:

- Every microservice connects strictly to its own dedicated MySQL database schema.
 - **Zero Cross-Database Joins**: No service is permitted to execute SQL joins across service boundaries.
 - **No Shared Tables**: Shared database entities are strictly prohibited. Information exchange between domains occurs exclusively via typed REST/OpenFeign requests or Kafka event messages.
 - **Independent Schema Evolution**: The `claims-service` database schema can be altered, migrated, or optimized with indexes without risking schema lock contention or regression in the `policy-service` or `customer-service`.
 - **Fault Containment**: If the `reporting_db` or `document_db` experiences high I/O saturation or downtime, critical customer onboarding and claim submission transactions continue uninterrupted.
-

6.2 Complete Microservice Database Schema Reference

1. `identity_db` (Identity Service)

- `users` :
 - `id` BIGINT AUTO_INCREMENT PRIMARY KEY
 - `username` VARCHAR(50) NOT NULL UNIQUE
 - `email` VARCHAR(100) NOT NULL UNIQUE
 - `password` VARCHAR(255) NOT NULL (BCrypt hash)
 - `first_name` VARCHAR(50) NOT NULL
 - `last_name` VARCHAR(50) NOT NULL
 - `enabled` BOOLEAN DEFAULT TRUE
 - `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP
 - `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
- `roles` :
 - `id` BIGINT AUTO_INCREMENT PRIMARY KEY
 - `name` VARCHAR(50) NOT NULL UNIQUE (e.g. `ROLE_CUSTOMER` , `ROLE_AGENT` , `ROLE_UNDERWRITER` , `ROLE_CLAIMS_OFFICER` , `ROLE_ADMIN`)
- `user_roles` :
 - `user_id` BIGINT NOT NULL, FK -> `users(id)`
 - `role_id` BIGINT NOT NULL, FK -> `roles(id)`
 - PRIMARY KEY (`user_id` , `role_id`)

2. `customer_db` (Customer Service)

- `customers` :
 - `id` BIGINT AUTO_INCREMENT PRIMARY KEY
 - `customer_code` VARCHAR(50) NOT NULL UNIQUE (e.g. `CUST-2026-0001`)
 - `first_name` VARCHAR(50) NOT NULL
 - `last_name` VARCHAR(50) NOT NULL
 - `email` VARCHAR(100) NOT NULL UNIQUE
 - `phone` VARCHAR(15) NOT NULL
 - `date_of_birth` DATE NOT NULL
 - `gender` VARCHAR(10) NOT NULL
 - `pan_number` VARCHAR(10)
 - `aadhaar_number` VARCHAR(12)
 - `kyc_status` VARCHAR(20) NOT NULL DEFAULT 'PENDING'
 - `created_at` , `updated_at` TIMESTAMP
- `customer_addresses` :
 - `id` BIGINT AUTO_INCREMENT PRIMARY KEY
 - `customer_id` BIGINT NOT NULL, FK -> `customers(id)`
 - `address_type` VARCHAR(20) NOT NULL
 - `street` VARCHAR(255) NOT NULL
 - `city` VARCHAR(100) NOT NULL
 - `state` VARCHAR(100) NOT NULL
 - `postal_code` VARCHAR(10) NOT NULL
 - `country` VARCHAR(50) NOT NULL DEFAULT 'India'
- `customer_nominees` :
 - `id` BIGINT AUTO_INCREMENT PRIMARY KEY
 - `customer_id` BIGINT NOT NULL, FK -> `customers(id)`
 - `nominee_name` VARCHAR(100) NOT NULL
 - `relationship` VARCHAR(50) NOT NULL
 - `date_of_birth` DATE NOT NULL
 - `allocation_percentage` DECIMAL(5,2) NOT NULL

3. `product_db` (Product & Plan Service)

- `products` :
 - `id` BIGINT AUTO_INCREMENT PRIMARY KEY
 - `product_code` VARCHAR(50) NOT NULL UNIQUE
 - `product_name` VARCHAR(100) NOT NULL
 - `product_type` VARCHAR(50) NOT NULL
 - `description` TEXT
 - `is_active` BOOLEAN DEFAULT TRUE

- **plans :**
- **id** BIGINT AUTO_INCREMENT PRIMARY KEY
- **product_id** BIGINT NOT NULL, FK -> **products(id)**
- **plan_code** VARCHAR(50) NOT NULL UNIQUE
- **plan_name** VARCHAR(100) NOT NULL
- **sum_insured** DECIMAL(15,2) NOT NULL
- **base_premium** DECIMAL(12,2) NOT NULL
- **min_age** INT NOT NULL
- **max_age** INT NOT NULL
- **is_active** BOOLEAN DEFAULT TRUE
- **plan_benefits :**
- **id** BIGINT AUTO_INCREMENT PRIMARY KEY
- **plan_id** BIGINT NOT NULL, FK -> **plans(id)**
- **benefit_name** VARCHAR(100) NOT NULL
- **benefit_code** VARCHAR(50) NOT NULL
- **limit_amount** DECIMAL(15,2)
- **waiting_period_days** INT DEFAULT 0
- **copay_percentage** DECIMAL(5,2) DEFAULT 0.00

4. **quotation_db (Quotation Service)**

- **quotes :**
- **id** BIGINT AUTO_INCREMENT PRIMARY KEY
- **quote_number** VARCHAR(50) NOT NULL UNIQUE
- **customer_id** BIGINT NOT NULL
- **plan_id** BIGINT NOT NULL
- **sum_insured** DECIMAL(15,2) NOT NULL
- **base_premium** DECIMAL(12,2) NOT NULL
- **tax_amount** DECIMAL(12,2) NOT NULL
- **total_premium** DECIMAL(12,2) NOT NULL
- **quote_status** VARCHAR(20) NOT NULL DEFAULT 'GENERATED'
- **expiry_date** DATE NOT NULL
- **created_at** TIMESTAMP
- **quote_members :**
- **id** BIGINT AUTO_INCREMENT PRIMARY KEY
- **quote_id** BIGINT NOT NULL, FK -> **quotes(id)**
- **relationship** VARCHAR(30) NOT NULL
- **age** INT NOT NULL
- **gender** VARCHAR(10) NOT NULL
- **tobacco_user** BOOLEAN DEFAULT FALSE

5. **risk_db** (Risk Assessment Service)

- **risk_assessments** :
 - **id** BIGINT AUTO_INCREMENT PRIMARY KEY
 - **assessment_number** VARCHAR(50) NOT NULL UNIQUE
 - **quote_id** BIGINT NOT NULL
 - **customer_id** BIGINT NOT NULL
 - **risk_score** INT NOT NULL (0 to 100)
 - **risk_grade** VARCHAR(20) NOT NULL (**LOW** , **MEDIUM** , **HIGH** , **DECLINED**)
 - **recommended_loading_pct** DECIMAL(5,2) DEFAULT 0.00
 - **assessment_status** VARCHAR(30) NOT NULL
 - **created_at** TIMESTAMP
- **risk_factors** :
 - **id** BIGINT AUTO_INCREMENT PRIMARY KEY
 - **assessment_id** BIGINT NOT NULL, FK -> **risk_assessments(id)**
 - **factor_name** VARCHAR(100) NOT NULL
 - **factor_type** VARCHAR(50) NOT NULL
 - **score_impact** INT NOT NULL
 - **details** VARCHAR(255)

6. **underwriting_db** (Underwriting Service)

- **underwriting_cases** :
 - **id** BIGINT AUTO_INCREMENT PRIMARY KEY
 - **case_number** VARCHAR(50) NOT NULL UNIQUE
 - **quote_id** BIGINT NOT NULL
 - **risk_assessment_id** BIGINT NOT NULL
 - **status** VARCHAR(30) NOT NULL (**APPROVED** , **REJECTED** , **PENDING_REVIEW** , **REFERRED**)
 - **approved_by** VARCHAR(100)
 - **premium_loading_percentage** DECIMAL(5,2) DEFAULT 0.00
 - **special_conditions** TEXT
 - **rejection_reason** VARCHAR(255)
 - **decided_at** TIMESTAMP

7. **policy_db** (Policy Service)

- **policies** :
 - **id** BIGINT AUTO_INCREMENT PRIMARY KEY
 - **policy_number** VARCHAR(50) NOT NULL UNIQUE
 - **customer_id** BIGINT NOT NULL
 - **plan_id** BIGINT NOT NULL
 - **quote_id** BIGINT NOT NULL

- `underwriting_case_id` BIGINT NOT NULL
- `status` VARCHAR(30) NOT NULL (`DRAFT` , `ISSUED` , `ACTIVE` , `EXPIRED` , `CANCELLED`)
- `start_date` DATE NOT NULL
- `end_date` DATE NOT NULL
- `sum_insured` DECIMAL(15,2) NOT NULL
- `total_premium` DECIMAL(12,2) NOT NULL
- `version` INT NOT NULL DEFAULT 0 (Optimistic Lock)
- `created_at` , `updated_at` TIMESTAMP
- **`policy_members` :**
- `id` BIGINT AUTO_INCREMENT PRIMARY KEY
- `policy_id` BIGINT NOT NULL, FK -> `policies(id)`
- `member_name` VARCHAR(100) NOT NULL
- `relationship` VARCHAR(30) NOT NULL
- `date_of_birth` DATE NOT NULL
- `sum_insured_share` DECIMAL(15,2)
- **`outbox_events` :**
- `id` BIGINT AUTO_INCREMENT PRIMARY KEY
- `aggregate_type` VARCHAR(50) NOT NULL
- `aggregate_id` VARCHAR(50) NOT NULL
- `event_type` VARCHAR(100) NOT NULL
- `payload` TEXT NOT NULL
- `status` VARCHAR(20) NOT NULL DEFAULT 'PENDING'
- `retry_count` INT DEFAULT 0
- `created_at` TIMESTAMP

8. `premium_db` (Premium Service)

- **`premium_schedules` :**
- `id` BIGINT AUTO_INCREMENT PRIMARY KEY
- `policy_id` BIGINT NOT NULL UNIQUE
- `policy_number` VARCHAR(50) NOT NULL
- `payment_frequency` VARCHAR(20) NOT NULL
- `total_installments` INT NOT NULL
- `total_amount` DECIMAL(12,2) NOT NULL
- `paid_amount` DECIMAL(12,2) DEFAULT 0.00
- `balance_amount` DECIMAL(12,2) NOT NULL
- `status` VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
- **`installments` :**
- `id` BIGINT AUTO_INCREMENT PRIMARY KEY
- `schedule_id` BIGINT NOT NULL, FK -> `premium_schedules(id)`

- `installment_number` INT NOT NULL
- `due_date` DATE NOT NULL
- `amount` DECIMAL(12,2) NOT NULL
- `grace_period_end_date` DATE NOT NULL
- `status` VARCHAR(20) NOT NULL DEFAULT 'PENDING'
- `paid_at` TIMESTAMP NULL

9. `payment_db` (Payment Service)

- `payments` :
- `id` BIGINT AUTO_INCREMENT PRIMARY KEY
- `transaction_reference` VARCHAR(100) NOT NULL UNIQUE
- `policy_id` BIGINT NOT NULL
- `installment_id` BIGINT NULL
- `amount` DECIMAL(12,2) NOT NULL
- `payment_method` VARCHAR(30) NOT NULL
- `payment_status` VARCHAR(30) NOT NULL
- `gateway_response_code` VARCHAR(50)
- `created_at` , `updated_at` TIMESTAMP
- `idempotency_records` :
- `id` BIGINT AUTO_INCREMENT PRIMARY KEY
- `idempotency_key` VARCHAR(100) NOT NULL UNIQUE
- `request_hash` VARCHAR(64) NOT NULL
- `response_body` TEXT
- `status` VARCHAR(20) NOT NULL
- `created_at` TIMESTAMP
- `outbox_events` : Transactional outbox table for payment events.

10. `provider_db` (Provider Service)

- `providers` :
- `id` BIGINT AUTO_INCREMENT PRIMARY KEY
- `provider_code` VARCHAR(50) NOT NULL UNIQUE
- `provider_name` VARCHAR(150) NOT NULL
- `provider_type` VARCHAR(50) NOT NULL
- `license_number` VARCHAR(100) NOT NULL UNIQUE
- `network_status` VARCHAR(30) NOT NULL DEFAULT 'IN_NETWORK'
- `tier` VARCHAR(20) NOT NULL DEFAULT 'TIER_1'
- `discount_percentage` DECIMAL(5,2) DEFAULT 0.00
- `is_active` BOOLEAN DEFAULT TRUE
- `created_at` TIMESTAMP
- `provider_departments` :

- `id` BIGINT AUTO_INCREMENT PRIMARY KEY
- `provider_id` BIGINT NOT NULL, FK -> `providers(id)`
- `department_name` VARCHAR(100) NOT NULL
- `contact_phone` VARCHAR(20)

11. `claims_db` (Claims Service - 11 Tables)

- `claims` :
 - `id` BIGINT AUTO_INCREMENT PRIMARY KEY
 - `claim_number` VARCHAR(50) NOT NULL UNIQUE
 - `policy_number` VARCHAR(50) NOT NULL
 - `customer_id` BIGINT NOT NULL
 - `provider_code` VARCHAR(50) NOT NULL
 - `claim_type` VARCHAR(30) NOT NULL (`CASHLESS` , `REIMBURSEMENT`)
 - `admission_date` DATE NOT NULL
 - `discharge_date` DATE NOT NULL
 - `claimed_amount` DECIMAL(15,2) NOT NULL
 - `approved_amount` DECIMAL(15,2) DEFAULT 0.00
 - `deductible_amount` DECIMAL(15,2) DEFAULT 0.00
 - `copay_amount` DECIMAL(15,2) DEFAULT 0.00
 - `settlement_status` VARCHAR(30) NOT NULL DEFAULT 'SUBMITTED'
 - `rejection_reason` VARCHAR(255)
 - `created_at` , `updated_at` TIMESTAMP
- `claim_diagnoses` : `id` , `claim_id` (FK), `icd10_code` , `description` , `is_primary`
- `claim_services` : `id` , `claim_id` (FK), `service_date` , `service_code` , `service_description` , `billed_amount` , `allowed_amount`
- `claim_documents` : `id` , `claim_id` (FK), `document_id` , `document_type` , `verified`
- `claim_validations` : `id` , `claim_id` (FK), `validation_stage` , `rule_name` , `passed` , `details`
- `claim_adjudications` : `id` , `claim_id` (FK), `total_billed` , `disallowed_amount` , `deductible_applied` , `copay_applied` , `net_approved` , `adjudicated_by` , `adjudicated_at`
- `explanation_of_benefits` : `id` , `claim_id` (FK), `eob_number` UNIQUE, `patient_responsibility` , `insurer_paid` , `notes` , `generated_at`
- `claim_payments` : `id` , `claim_id` (FK), `payment_reference` , `payment_amount` , `disbursement_status` , `disbursed_at`
- `idempotency_records` : Deduplication store for claims
- `outbox_events` : Outbox table for `claim-events`
- `audit_records` : Entity audit change log

12. `document_db` (Document Service)

- `documents` :
 - `id` BIGINT AUTO_INCREMENT PRIMARY KEY

- `document_code` VARCHAR(50) NOT NULL UNIQUE
- `file_name` VARCHAR(255) NOT NULL
- `file_type` VARCHAR(100) NOT NULL
- `file_size` BIGINT NOT NULL
- `storage_path` VARCHAR(500) NOT NULL
- `entity_type` VARCHAR(50) NOT NULL
- `entity_id` BIGINT NOT NULL
- `uploaded_at` TIMESTAMP

13. `notification_db` & `reporting_db`

- `notification_db.notifications`: `id`, `recipient_email`, `recipient_phone`, `notification_type`, `subject`, `message_body`, `event_source`, `delivery_status`, `sent_at`.
 - `reporting_db.daily_kpi_summaries`: `id`, `summary_date` UNIQUE, `total_policies_issued`, `total_premium_collected`, `total_claims_submitted`, `total_claims_approved`, `total_claims_amount_paid`, `loss_ratio`.
 - `reporting_db.claim_analytics`: `id`, `claim_number`, `policy_number`, `provider_code`, `claimed_amount`, `approved_amount`, `adjudication_time_seconds`.
-

7. REST API ARCHITECTURE

7.1 REST Principles & Engineering Standards

All HIMS microservices adhere strictly to REST architectural constraints:

1. **Resource Identification via URIs:** Plural nouns represent collections (e.g. `/api/policies`, `/api/claims`, `/api/customers`).

2. **Standard HTTP Verbs:**

- `GET` : Safe, idempotent read operations.
- `POST` : Resource creation and command execution.
- `PUT` : Idempotent full state updates or lifecycle state transitions (e.g. `/approve`).
- `DELETE` : Resource removal or soft-deletion.

3. **Standard HTTP Status Codes:**

- `200 OK` : Successful retrieval or modification.
- `201 Created` : Successful resource creation (with `Location` header or created object body).
- `400 Bad Request` : JSR-380 validation failure or malformed payload.
- `401 Unauthorized` : Missing, expired, or invalid JWT token.
- `403 Forbidden` : Authenticated user lacks required role/authority.
- `404 Not Found` : Resource identifier does not exist.
- `409 Conflict` : Duplicate business key or optimistic locking version conflict.
- `500 Internal Server Error` : Unhandled infrastructure failure.

7.2 Standardized API Response Wrapper

All endpoints return an immutable, standardized response envelope:

```
{
  "success": true,
  "message": "Policy issued successfully",
  "data": {
    "id": 1042,
    "policyNumber": "POL-2026-0089",
    "status": "ISSUED",
    "sumInsured": 500000.00,
    "totalPremium": 14500.00
  },
  "timestamp": "2026-09-30T10:15:30Z"
}
```

7.3 Global Exception Handling & Error Envelope

Global exceptions are intercepted by `@RestControllerAdvice` classes in each service, normalizing errors into a secure schema:

```
{
  "success": false,
  "errorCode": "VALIDATION_FAILED",
  "message": "Input validation failed for 2 field(s)",
  "errors": {
    "panNumber": "PAN number must match uppercase 10-character pattern [A-Z]{5}[0-9]{4}[A-Z]{1}",
    "phone": "Phone number must be exactly 10 digits"
  },
  "timestamp": "2026-09-30T10:15:32Z",
  "path": "/api/customers"
}
```

7.4 Representative API Implementation Examples

Example 1: Issue Policy API (`policy-service`)

- **Verb & Path:** `POST /api/policies/issue`
- **Authorization:** `hasAnyRole('ROLE_UNDERWRITER', 'ROLE_AGENT', 'ROLE_ADMIN')`
- **Request Body:**

```
{
  "quoteId": 204,
  "underwritingCaseId": 102,
  "paymentFrequency": "ANNUAL"
}
```

- **Response (`201 Created`):**

```
{
  "success": true,
  "message": "Policy successfully issued and queued for premium schedule generation",
  "data": {
    "policyId": 89,
    "policyNumber": "POL-2026-0089",
    "status": "ISSUED",
    "startDate": "2026-10-01",
    "endDate": "2027-09-30",
    "sumInsured": 500000.00,
    "totalPremium": 14500.00
  }
}
```

Example 2: Submit Healthcare Claim (`claims-service`)

- **Verb & Path:** `POST /api/claims`
- **Headers:** `X-Idempotency-Key: 7b8e1f0e-3c9a-412d-b152-32a893c52a01`
- **Request Body:**

```
{
  "policyNumber": "POL-2026-0089",
  "customerId": 15,
  "providerCode": "HOSP-MAX-001",
  "claimType": "CASHLESS",
  "admissionDate": "2026-10-15",
  "dischargeDate": "2026-10-18",
  "claimedAmount": 45000.00,
  "diagnoses": [
    {
      "icd10Code": "J18.9",
      "description": "Pneumonia, unspecified organism",
      "isPrimary": true
    }
  ],
  "services": [
    {
      "serviceDate": "2026-10-15",
      "serviceCode": "ICU-BED",
      "serviceDescription": "Intensive Care Bed Charge",
      "billedAmount": 30000.00
    },
    {
      "serviceDate": "2026-10-16",
      "serviceCode": "MED-ANTIBIOTIC",
      "serviceDescription": "Intravenous Antibiotics",
      "billedAmount": 15000.00
    }
  ]
}
```

- **Response (201 Created):**

```
{
  "success": true,
  "message": "Claim submitted and automated adjudication initiated",
  "data": {
    "claimId": 501,
    "claimNumber": "CLM-2026-0501",
    "settlementStatus": "SUBMITTED",
    "claimedAmount": 45000.00
  }
}
```

8. KAFKA EVENT-DRIVEN ARCHITECTURE

8.1 Why Apache Kafka in HIMIS?

In an enterprise insurance platform, tight synchronous coupling across all services creates catastrophic brittleness:

1. **Temporal Decoupling:** If the `notification-service` is restarting for an upgrade, policy issuance must NOT fail. The event sits durably in the `policy-events` topic until the consumer comes back online.
2. **Transactional Outbox Guarantee:** By writing domain events to a local relational `outbox_events` table within the same database transaction as the business entity, HIMIS guarantees **At-Least-Once Delivery** with zero data loss even during sudden power failure.
3. **High Throughput & Replayability:** Apache Kafka's partitioned commit log allows analytical consumers in `reporting-service` to reprocess event streams from offset 0 to recompute historical metrics.

8.2 Standard HIMIS Event Envelope Schema

Every event transmitted through Kafka adheres to an enterprise envelope:

```
{
  "eventId": "e9b2512a-89a1-43ef-b72e-8a2bf1893c01",
  "eventType": "PolicyIssuedEvent",
  "eventVersion": "1.0",
  "timestamp": "2026-09-30T10:15:30.125Z",
  "correlationId": "corr-78a9c2b4-5211",
  "aggregateId": "POL-2026-0089",
  "aggregateType": "POLICY",
  "payload": {
    "policyId": 89,
    "policyNumber": "POL-2026-0089",
    "customerId": 15,
    "planId": 3,
    "totalPremium": 14500.00,
    "paymentFrequency": "ANNUAL",
    "startDate": "2026-10-01",
    "endDate": "2027-09-30"
  }
}
```

8.3 Kafka Topics & Event Matrix

Kafka Topic	Event Name	Producer Service	Consumer Service(s)
policy-events	PolicyIssuedEvent	policy-service	premium-service , notification-service , reporting-service
policy-events	PremiumScheduleFailedEvent	premium-service	policy-service (PolicySagaCompensationConsumer
policy-events	PolicyActivatedEvent	policy-service	notification-service , reporting-service
premium-events	PremiumPaidEvent	payment-service	policy-service , premium-service , notification-service
claim-events	ClaimSubmittedEvent	claims-service	notification-service , reporting-service
claim-events	ClaimApprovedEvent	claims-service	notification-service , reporting-service
claim-events	ClaimRejectedEvent	claims-service	notification-service , reporting-service
risk-events	RiskAssessmentCompletedEvent	risk-service	underwriting-service

Kafka Topic	Event Name	Producer Service	Consumer Service(s)
underwriting-events	UnderwritingApprovedEvent	underwriting-service	policy-service

8.4 REST Synchronous vs Kafka Asynchronous Architecture Comparison

Dimension	REST / OpenFeign Synchronous	Apache Kafka Asynchronous
Communication Style	Request / Response (Point-to-Point)	Publish / Subscribe (Event-Driven)
Coupling	High temporal coupling (Both caller and receiver must be alive)	Low temporal coupling (Producer finishes immediately)
Latency	Immediate return of computed data	Eventual consistency (Milliseconds to seconds)
Use Case in HIMS	Claim eligibility checks (Needs instant YES/NO answer from PolicyService)	Premium schedule creation, payment notifications, BI reporting
Failure Behavior	Circuit Breaker fallback or immediate HTTP 500 error	Event buffered safely in Kafka topic for automatic consumer retry

9. SECURITY ARCHITECTURE

9.1 Enterprise Security Architecture Model

HIMS implements a defense-in-depth, zero-trust security perimeter combining edge gateway token authentication with decentralized microservice role enforcement.

```
[ Angular 17 UI ]
| HTTP Request + Authorization: Bearer <JWT>
▼
[ Spring Cloud Gateway (8080) ]
| 1. Verifies JWT HMAC-SHA256 signature
| 2. Checks expiration
| 3. Injects claims into downstream headers:
|   - X-User-Id: 42
|   - X-Username: underwriter_john
|   - X-User-Roles: ROLE_UNDERWRITER
▼
[ Downstream Microservice (e.g. policy-service) ]
| 1. JwtAuthenticationFilter parses Bearer token
| 2. Populates Spring SecurityContextHolder
| 3. Method-level enforcement: @PreAuthorize("hasRole('ROLE_UNDERWRITER')")
```

9.2 JSON Web Token (JWT) Specification

Authentication tokens are issued by **identity-service** upon verified login and signed using HMAC-SHA256 (**HS256**) with an enterprise 256-bit secret key.

JWT Structure

- **Header:**

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- **Payload Claims:**

```
{
  "sub": "underwriter_john",
  "userId": 42,
  "email": "john.underwriter@hims-enterprise.com",
  "roles": ["ROLE_UNDERWRITER"],
  "iat": 1727690000,
  "exp": 1727776400
}
```

- **Signature:** `HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secretKey)`

9.3 Role-Based Access Control (RBAC) Matrix

User Role	Implementation Status	Permitted Operations / Endpoints
ROLE_ADMIN	✓ IMPLEMENTED	Full CRUD access to all services, user role provisioning, system settings, global provider creation, and executive reporting.
ROLE_CUSTOMER	✓ IMPLEMENTED	Self-registration, view available plans, generate quotes, view owned policies, pay premiums, upload claim documents, submit claims, download own EOBs.
ROLE_AGENT	✓ IMPLEMENTED	Create and manage quotes for clients, register customer profiles, initiate policy binding requests.
ROLE_UNDERWRITER	✓ IMPLEMENTED	Review medical risk assessments, evaluate policy applications, set premium loading percentages, approve or decline underwriting cases (<code>PUT /api/underwriting/{id}/approve</code>).
ROLE_CLAIMS_OFFICER	✓ IMPLEMENTED	Review submitted claims, inspect automated validation rule results, execute adjudication calculations, approve or reject claims, authorize settlement disbursements.
ROLE_FINANCE_OFFICER	⚠ PARTIALLY IMPLEMENTED	View payment transactions, inspect billing schedules, download financial reconciliation reports (largely merged into ROLE_ADMIN in current UI).

9.4 Method-Level Authorization Examples

Spring Security's `@EnableMethodSecurity(prePostEnabled = true)` protects controller methods:

```
// Underwriting Service: Only licensed underwriters can approve cases
@PutMapping("/{id}/approve")
@PreAuthorize("hasAnyRole('ROLE_UNDERWRITER', 'ROLE_ADMIN')")
public ResponseEntity<ApiResponse<UnderwritingCaseDto>> approveCase(@PathVariable Long id) {
    return ResponseEntity.ok(underwritingService.approveCase(id));
}

// Claims Service: Only claims officers can execute manual override adjudications
@PostMapping("/{id}/adjudicate")
@PreAuthorize("hasAnyRole('ROLE_CLAIMS_OFFICER', 'ROLE_ADMIN')")
public ResponseEntity<ApiResponse<AdjudicationResultDto>> adjudicateClaim(@PathVariable Long id) {
    return ResponseEntity.ok(claimAdjudicationService.adjudicateClaim(id));
}
```

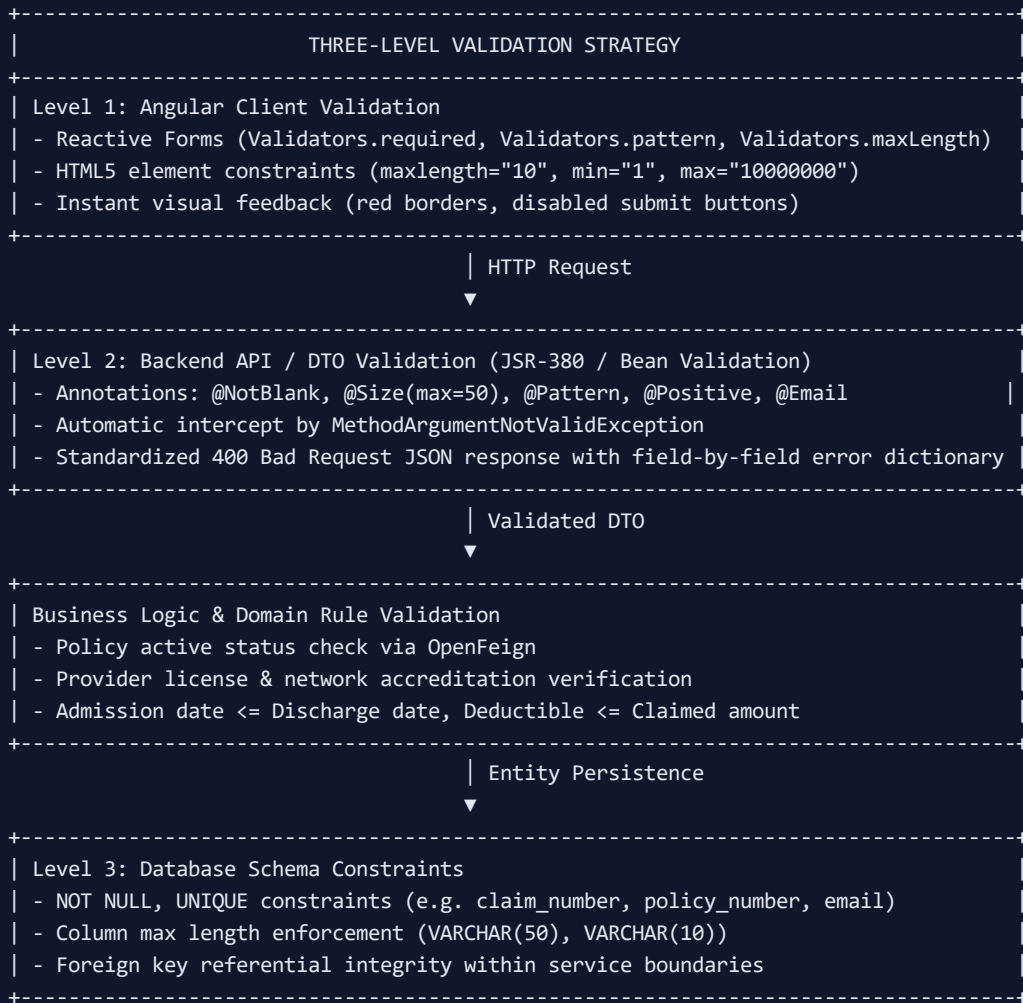
9.5 Password Security & Data Protection

- **BCrypt Hashing:** All user passwords stored in `identity_db.users` are hashed using `BCryptPasswordEncoder` with an internal cost factor of 10. Passwords are never logged or returned in DTOs.

- **Stateless Session Management:** Microservices operate with `SessionCreationPolicy.STATELESS` , eliminating HTTP session hijacking vulnerabilities.
-

10. VALIDATION ARCHITECTURE

HIMS enforces a **Three-Level Validation Architecture** backed by strict **Domain Business Rules** to prevent data corruption and ensure zero garbage data reaches persistence.



10.1 Level 1: Frontend Angular Validation

In the Angular client (`frontend/hims-ui`), all user inputs are governed by Angular Reactive Forms with HTML5 bounds:

```
this.claimForm = this.fb.group({
  policyNumber: ['', [Validators.required, Validators.pattern('^POL-[0-9]{4}-[0-9]{4}$'), Validators.maxLength(10)],
  providerCode: ['', [Validators.required, Validators.pattern('^A-Z0-9-[0-9]{3,5}$'), Validators.maxLength(50)]],
  claimedAmount: [null, [Validators.required, Validators.min(1), Validators.max(5000000)]],
  admissionDate: ['', [Validators.required]],
  dischargeDate: ['', [Validators.required]]
});
```

- Form submit buttons are bound to `[disabled]="!claimForm.valid"`, preventing accidental submission of empty or malformed inputs.
- All input fields enforce explicit `maxlength` HTML attributes to prevent buffer flooding when users paste large payloads.

10.2 Level 2: Backend DTO JSR-380 Annotations

All incoming REST DTOs are validated at the controller boundary using `@Valid`:

```
public class CustomerRegistrationDto {

    @NotBlank(message = "First name is mandatory")
    @Size(min = 2, max = 50, message = "First name must be between 2 and 50 characters")
    @Pattern(regexp = "[a-zA-Z\\s]+$", message = "First name can only contain alphabetic characters")
    private String firstName;

    @NotBlank(message = "Email is mandatory")
    @Email(message = "Email must be a valid email format")
    @Size(max = 100, message = "Email cannot exceed 100 characters")
    private String email;

    @NotBlank(message = "Phone number is mandatory")
    @Pattern(regexp = "[0-9]{10}$", message = "Phone number must be exactly 10 digits")
    private String phone;

    @NotBlank(message = "PAN number is mandatory")
    @Pattern(regexp = "[A-Z]{5}[0-9]{4}[A-Z]{1}$", message = "PAN must follow standard format (e.g. ABCDE1234F)")
    private String panNumber;

    @NotNull(message = "Date of birth is mandatory")
    @Past(message = "Date of birth must be in the past")
    private LocalDate dateOfBirth;

}
```

10.3 Level 3: Database Schema Validation

At the persistence tier, MySQL constraints guarantee structural integrity:

- `NOT NULL` on critical identifiers (`policy_number` , `claim_number` , `claimed_amount`).
- `UNIQUE` indexes on business keys preventing duplicate accounts or transactions.
- Length specifications (`VARCHAR(10)` for PAN, `VARCHAR(12)` for Aadhaar, `VARCHAR(50)` for codes).

10.4 Business Domain Validations

Beyond syntactic checks, services execute domain-specific semantic validations:

1. **Admission Date Temporal Consistency:** `dischargeDate >= admissionDate` .
2. **Policy Coverage Window:** `policy.startDate <= admissionDate <= policy.endDate` .
3. **Policy Active Status:** Only policies with `status == 'ACTIVE'` are eligible for claim adjudication.

4. **Duplicate Claim Detection:** No two claims with identical patient, provider, and admission date can be submitted.

11. SAGA PATTERN & TRANSACTION MANAGEMENT

11.1 The Distributed Transaction Problem

In a microservices architecture, a single business workflow—such as policy issuance and billing schedule creation—spans multiple independent databases (`policy_db` , `premium_db` , `payment_db`). Traditional ACID transactions with 2-Phase Commit (2PC) are unsuitable because:

- They cause heavy network latency and distributed deadlocks.
- They severely degrade availability (violating CAP theorem principles).
- MySQL 2PC across microservices introduces tight coupling.

To solve this, HIMS implements **Choreography-based Sagas** combined with the **Transactional Outbox Pattern**.



11.2 Compensating Transaction Code Walkthrough

In `policy-service` , compensation is handled by `PolicySagaCompensationConsumer` :

```

@Component
@Slf4j
public class PolicySagaCompensationConsumer {

    @Autowired
    private PolicyRepository policyRepository;

    @KafkaListener(topics = "policy-events", groupId = "policy-saga-group")
    public void handlePolicyEvents(String message) {
        EventEnvelope event = parseEnvelope(message);
        if ("PremiumScheduleFailedEvent".equals(event.getEventType())) {
            String policyNumber = event.getAggregateId();
            log.warn("SAGA COMPENSATION TRIGGERED for Policy: {}", policyNumber);
            Policy policy = policyRepository.findByPolicyNumber(policyNumber)
                .orElseThrow(() -> new ResourceNotFoundException("Policy not found"));

            // Compensating action: cancel policy and log failure reason
            policy.setStatus(PolicyStatus.CANCELLED);
            policy.setUpdatedReason("SAGA Compensation: Premium schedule creation failed");
            policyRepository.save(policy);
            log.info("Policy {} successfully rolled back to CANCELLED state.", policyNumber);
        }
    }
}

```

11.3 Transactional Outbox Implementation

Implemented in `claims-service`, `payment-service`, and `policy-service`:

1. **Local Transaction:** The business entity and the `outbox_events` row are committed together:

```

@Transactional
public Policy issuePolicy(IssuePolicyRequest request) {
    Policy policy = createPolicyEntity(request);
    policyRepository.save(policy);

    OutboxEvent outbox = OutboxEvent.builder()
        .aggregateType("POLICY")
        .aggregateId(policy.getPolicyNumber())
        .eventType("PolicyIssuedEvent")
        .payload(objectMapper.writeValueAsString(policy))
        .status("PENDING")
        .retryCount(0)
        .build();
    outboxRepository.save(outbox);
    return policy;
}

```

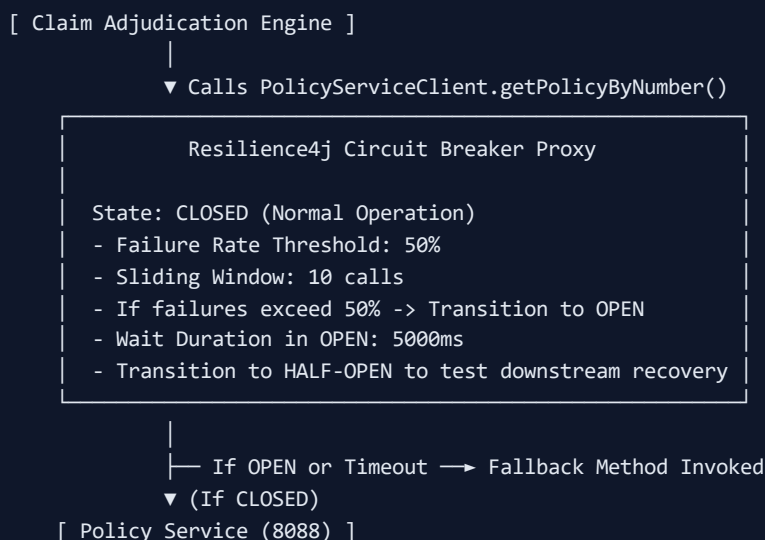
2. **Scheduled Relayer:** A scheduled background thread polls for `PENDING` records:

```
@Scheduled(fixedDelay = 5000)
public void publishPendingOutboxEvents() {
    List<OutboxEvent> events = outboxRepository.findByStatus("PENDING");
    for (OutboxEvent event : events) {
        try {
            kafkaTemplate.send("policy-events", event.getAggregateId(), event.getPayload());
            event.setStatus("PUBLISHED");
            outboxRepository.save(event);
        } catch (Exception ex) {
            event.setRetryCount(event.getRetryCount() + 1);
            if (event.getRetryCount() >= 5) event.setStatus("FAILED");
            outboxRepository.save(event);
        }
    }
}
```

12. RESILIENCE AND FAULT TOLERANCE

12.1 Circuit Breaker & Retry Patterns (Resilience4j)

Distributed microservices are susceptible to cascading network latencies and transient downstream service crashes. HIMS integrates **Resilience4j** in the `claims-service` to protect synchronous OpenFeign invocations targeting `policy-service` and `provider-service`.



12.2 Configuration in `config-repo/claims-service.yml`

```
resilience4j:
  circuitbreaker:
    instances:
      policyServiceBreaker:
        slidingWindowSize: 10
        minimumNumberOfCalls: 5
        failureRateThreshold: 50
        waitDurationInOpenState: 5000ms
        permittedNumberOfCallsInHalfOpenState: 3
        automaticTransitionFromOpenToHalfOpenEnabled: true
      providerServiceBreaker:
        slidingWindowSize: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 5000ms
  retry:
    instances:
      policyServiceRetry:
        maxAttempts: 3
        waitDuration: 1000ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2.0
```

12.3 Feign Client with CircuitBreaker & Fallback Implementation

```
@FeignClient(name = "policy-service", fallback = PolicyServiceFallback.class)
public interface PolicyServiceClient {

    @GetMapping("/api/policies/number/{policyNumber}")
    @CircuitBreaker(name = "policyServiceBreaker", fallbackMethod = "getPolicyFallback")
    @Retry(name = "policyServiceRetry")
    ApiResponse<PolicyDto> getPolicyByNumber(@PathVariable("policyNumber") String policyNumber);

    default ApiResponse<PolicyDto> getPolicyFallback(String policyNumber, Throwable t) {
        log.error("Fallback invoked for PolicyServiceClient: {}. Reason: {}", policyNumber, t.getMessage());
        return ApiResponse.<PolicyDto>builder()
            .success(false)
            .message("Policy Service is temporarily unavailable. Claim queued for offline verification.")
            .data(null)
            .build();
    }
}
```

13. IDEMPOTENCY IMPLEMENTATION

13.1 The Double-Submit & Duplicate Payment Dilemma

In distributed financial and healthcare systems, network instability and aggressive client retries can lead to catastrophic duplicate operations:

- A user clicks "Pay Premium" twice in rapid succession.
- A network drop occurs after the payment gateway debits the card but before the HTTP 200 response reaches the browser.
- A hospital automated interface resubmits an emergency cashless claim after a 5-second socket timeout.

To prevent duplicate charges or duplicate claim adjudications, HMS implements **Header-driven Distributed Idempotency** in both `payment-service` and `claims-service`.

```
[ Client / Angular UI ]
|
| HTTP POST /api/payments/process
| Header: X-Idempotency-Key: "c6a1b2d3-9f8e-4a7b-b892-123456789abc"
| Body: { policyId: 89, amount: 14500.00 }
|
▼
[ Payment Service / Idempotency Filter ]
|
| 1. Compute SHA-256 Hash of Request Body
| 2. Query idempotency_records WHERE idempotency_key = key
|
| → [Record NOT FOUND]:
|   - Insert record (status='PROCESSING', request_hash=hash)
|   - Execute payment transaction against gateway
|   - Save payment entity
|   - Update record (status='COMPLETED', response_body=jsonResponse)
|   - Return 201 Created to caller
|
| → [Record FOUND with status='PROCESSING']:
|   - Reject with HTTP 409 Conflict ("Concurrent request in progress")
|
| → [Record FOUND with status='COMPLETED']:
|   - Compare incoming request_hash with stored request_hash
|   - If hashes MATCH: Return cached response_body with HTTP 200 OK (Zero duplicate charge!)
|   - If hashes DIFFER: Reject with HTTP 400 Bad Request ("Key reused for different payload")
```

13.2 Database Schema: `idempotency_records`

```
CREATE TABLE idempotency_records (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    idempotency_key VARCHAR(100) NOT NULL UNIQUE,
    request_hash VARCHAR(64) NOT NULL,
    response_body TEXT NULL,
    status VARCHAR(20) NOT NULL, -- 'PROCESSING', 'COMPLETED'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

13.3 Implementation Code Walkthrough

```
@Service
@Transactional
public class IdempotentPaymentService {

    @Autowired
    private IdempotencyRecordRepository idempotencyRepository;

    public PaymentResponse processIdempotentPayment(String idempotencyKey, PaymentRequest request) {
        String requestHash = Sha256Utils.hash(objectMapper.writeValueAsString(request));

        Optional<IdempotencyRecord> existing = idempotencyRepository.findByIdempotencyKey(idempotencyKey);
        if (existing.isPresent()) {
            IdempotencyRecord record = existing.get();
            if ("PROCESSING".equals(record.getStatus())) {
                throw new ConcurrentRequestException("Payment is already being processed for key: " + idempotencyKey);
            }
            if (!record.getRequestHash().equals(requestHash)) {
                throw new InvalidIdempotencyKeyException("Idempotency key reused with different request payload");
            }
            // Return cached response directly
            return objectMapper.readValue(record.getResponseBody(), PaymentResponse.class);
        }

        // Lock key with PROCESSING status
        IdempotencyRecord newRecord = IdempotencyRecord.builder()
            .idempotencyKey(idempotencyKey)
            .requestHash(requestHash)
            .status("PROCESSING")
            .build();
        idempotencyRepository.save(newRecord);

        // Execute core payment transaction
        PaymentResponse response = executeGatewayCharge(request);

        // Cache completed result
        newRecord.setStatus("COMPLETED");
        newRecord.setResponseBody(objectMapper.writeValueAsString(response));
        idempotencyRepository.save(newRecord);

        return response;
    }
}
```

14. AUDIT AND LOGGING ARCHITECTURE

14.1 Enterprise Audit Trail Strategy

Regulatory frameworks (such as HIPAA and Insurance Regulatory Development Authorities) require strict auditing of all operational and financial events. HIMS implements automated auditing at both the **database level** and the **application aspect level**.

Database Entity Auditing

Persistent entities inherit from an abstract base class or implement audit fields:

- `created_at` : Timestamp of initial creation.
- `updated_at` : Automatically updated by database trigger or Hibernate `@UpdateTimeStamp` .
- `created_by` / `updated_by` : User ID or username resolved from the Spring Security context.
- `version` : Optimistic locking sequence number.

`claims-db.audit_records` Schema

The `claims-service` maintains a dedicated audit table:

```
CREATE TABLE audit_records (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  entity_name VARCHAR(50) NOT NULL,  
  entity_id BIGINT NOT NULL,  
  action VARCHAR(50) NOT NULL, -- 'SUBMIT', 'VALIDATE', 'ADJUDICATE', 'APPROVE', 'REJECT'  
  performed_by VARCHAR(100) NOT NULL,  
  previous_state TEXT,  
  new_state TEXT,  
  ip_address VARCHAR(50),  
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

14.2 Spring AOP Audit & Execution Profiler

```

@Aspect
@Component
@Slf4j
public class ServiceAuditAspect {

    @Around("execution(* com.hims..service..*(..))")
    public Object auditServiceMethods(ProceedingJoinPoint joinPoint) throws Throwable {
        long startTime = System.currentTimeMillis();
        String methodName = joinPoint.getSignature().toShortString();
        String username = SecurityContextHolder.getContext().getAuthentication() != null ?
            SecurityContextHolder.getContext().getAuthentication().getName() : "ANONYMOUS";

        log.info("[AUDIT-START] User='{}', Method='{}'", username, methodName);
        try {
            Object result = joinPoint.proceed();
            long duration = System.currentTimeMillis() - startTime;
            log.info("[AUDIT-SUCCESS] User='{}', Method='{', Duration={}'ms", username, methodName, duration);
            return result;
        } catch (Exception ex) {
            long duration = System.currentTimeMillis() - startTime;
            log.error("[AUDIT-FAILURE] User='{', Method='{', Duration={}'ms, Error='{'", username, methodName,
                throw ex;
        }
    }
}

```

14.3 Distributed Correlation Tracing with MDC

When requests arrive at the `api-gateway`, a unique `X-Correlation-Id` UUID is generated and attached to request headers. Downstream microservices register this ID in Logback's **Mapped Diagnostic Context (MDC)**:

```

2026-09-30 10:15:30.120 [http-nio-8092-exec-1] [corr-78a9c2b4] INFO com.hims.claims.ClaimController - Received
2026-09-30 10:15:30.145 [http-nio-8092-exec-1] [corr-78a9c2b4] INFO com.hims.claims.PolicyClient - Calling Pol

```

This enables DevOps and developers to grep the entire cluster for `corr-78a9c2b4` to trace a single transaction across Gateway, Policy, Provider, Claims, and Kafka logs.

15. OPENFEIGN COMMUNICATION

15.1 Declarative Synchronous RPC via OpenFeign

Spring Cloud OpenFeign eliminates repetitive HTTP boilerplate code by allowing developers to define declarative Java interfaces annotated with Spring MVC mappings. In HMS, OpenFeign works hand-in-hand with Netflix Eureka for client-side load balancing.

COMPLETE OPENFEIGN CLIENT DIRECTORY		
Calling Microservice	Feign Client Interface	Target Microservice
claims-service	PolicyServiceClient	policy-service (8088)
claims-service	ProviderServiceClient	provider-service (8091)
policy-service	CustomerClient	customer-service (8083)
policy-service	ProductClient	product-plan-service(8084)
policy-service	QuotationClient	quotation-service (8085)
policy-service	PremiumClient	premium-service (8089)
policy-service	PaymentClient	payment-service (8090)
premium-service	ProductPlanClient	product-plan-service(8084)
premium-service	QuotationClient	quotation-service (8085)
quotation-service	CustomerClient	customer-service (8083)
quotation-service	ProductClient	product-plan-service(8084)
risk-service	CustomerClient	customer-service (8083)
risk-service	QuotationClient	quotation-service (8085)
underwriting-service	QuotationClient	quotation-service (8085)
underwriting-service	RiskAssessmentClient	risk-service (8086)

15.2 Representative Feign Client Definitions

1. claims-service -> PolicyServiceClient

```
@FeignClient(name = "policy-service", configuration = FeignClientConfiguration.class)
public interface PolicyServiceClient {

    @GetMapping("/api/policies/number/{policyNumber}")
    ApiResponse<PolicyDto> getPolicyByNumber(@PathVariable("policyNumber") String policyNumber);

    @GetMapping("/api/policies/{id}/eligibility")
    ApiResponse<PolicyEligibilityDto> checkEligibility(@PathVariable("id") Long id, @RequestParam("date") String date);
}
```

2. claims-service -> ProviderServiceClient

```
@FeignClient(name = "provider-service", configuration = FeignClientConfiguration.class)
public interface ProviderServiceClient {

    @GetMapping("/api/providers/code/{providerCode}")
    ApiResponse<ProviderDto> getProviderByCode(@PathVariable("providerCode") String providerCode);
}
```

3. quotation-service -> CustomerClient & ProductClient

```
@FeignClient(name = "customer-service")
public interface CustomerClient {
    @GetMapping("/api/customers/{id}")
    ApiResponse<CustomerDto> getCustomerById(@PathVariable("id") Long id);
}

@FeignClient(name = "product-plan-service")
public interface ProductClient {
    @GetMapping("/api/plans/{id}")
    ApiResponse<PlanDto> getPlanById(@PathVariable("id") Long id);
}
```

15.3 Global Feign Configuration & Error Decoding

To prevent uncaught 500 runtime exceptions when a downstream service returns a 404 or 400, HIMS implements a custom `ErrorDecoder` :

```
public class CustomFeignErrorDecoder implements ErrorDecoder {
    @Override
    public Exception decode(String methodKey, Response response) {
        if (response.status() == 404) {
            return new ResourceNotFoundException("Remote entity not found via Feign call: " + methodKey);
        }
        if (response.status() == 400) {
            return new BadRequestException("Remote service reported bad request: " + methodKey);
        }
        return new ServiceUnavailableException("Downstream service communication failure: " + response.status());
    }
}
```

16. FRONTEND ARCHITECTURE

16.1 Angular 17 Application Structure

The HIMS client is a high-performance Single Page Application (SPA) built with Angular 17 and TypeScript, located in `frontend/hims-ui`. It employs a 3-tier modular directory layout:

```
src/app/
├── core/                # Singleton services, interceptors, authentication guards
│   ├── auth/           # AuthService, TokenStorageService, AuthGuard, RoleGuard
│   ├── interceptors/   # JwtInterceptor, ErrorInterceptor
│   ├── models/         # Strongly-typed TypeScript interfaces matching Java DTOs
│   └── services/       # ApiService, NotificationService, UtilityService
├── features/           # Domain-specific feature modules with routed views
│   ├── auth/           # Login, Register, Forgot Password
│   ├── customer/       # Customer registration, Profile, KYC upload
│   ├── product-plan/   # Product catalog, Plan comparison cards
│   ├── quotation/      # Quote wizard, actuarial loading preview
│   ├── risk/           # Medical disclosures, BMI calculator, risk score card
│   ├── underwriting/   # Underwriter case approval dashboard & loading adjuster
│   ├── policy/         # Policy list, Policy issuance wizard, e-Card download
│   ├── premium/        # Installment schedules, Due date timeline, receipt viewer
│   ├── payment/        # Mock gateway payment checkout modal, receipt view
│   ├── provider/       # Hospital directory search, In-network map/filter
│   ├── claim/          # Claim submission form, 6-stage status tracker, EOB viewer
│   ├── document/       # Document repository, drag-and-drop file uploader
│   ├── notification/   # Notification bell, alert center
│   ├── report/         # Executive analytics charts, claim settlement KPIs
│   └── user-management/ # Admin user provisioning & role assignment
└── shared/            # Reusable UI components, pipes, directives, modals
```

16.2 HTTP Interceptors Pipeline

1. `JwtInterceptor`

Automatically intercepts every outgoing HTTP request to `/api/` and injects the `Authorization: Bearer <token>` header retrieved from `TokenStorageService`.

```

@Injectables()
export class JwtInterceptor implements HttpInterceptor {
  constructor(private tokenStorage: TokenStorageService) {}

  intercept(request: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = this.tokenStorage.getToken();
    if (token) {
      request = request.clone({
        setHeaders: { Authorization: `Bearer ${token}` }
      });
    }
    return next.handle(request);
  }
}

```

2. **ErrorInterceptor**

Catches backend error responses globally. If an HTTP 401 is received, it clears the token storage and routes the user to `/auth/login`. If an HTTP 400 validation error occurs, it parses the field errors and renders visual alerts.

16.3 Route Guards Architecture

- **AuthGuard**: Protects all private routes. Checks if a valid, unexpired token exists in local storage.
- **RoleGuard**: Verifies that the authenticated user possesses the specific role required by the route configuration:

```

const routes: Routes = [
  {
    path: 'underwriting',
    component: UnderwritingDashboardComponent,
    canActivate: [AuthGuard, RoleGuard],
    data: { expectedRoles: ['ROLE_UNDERWRITER', 'ROLE_ADMIN'] }
  },
  {
    path: 'claims/adjudication',
    component: ClaimAdjudicationComponent,
    canActivate: [AuthGuard, RoleGuard],
    data: { expectedRoles: ['ROLE_CLAIMS_OFFICER', 'ROLE_ADMIN'] }
  }
];

```

17. DESIGN PATTERNS USED IN HIMS

HIMS incorporates proven enterprise software design patterns across its architectural, creational, structural, and behavioral tiers.

17.1 Architectural & Distributed Patterns

Pattern	Where Used	Implementation Details & Purpose
Transactional Outbox	<code>claims-service</code> , <code>policy-service</code> , <code>payment-service</code>	Solves the dual-write problem by saving domain events in a relational <code>outbox_events</code> table in the same ACID transaction as the business entity. A background scheduled poller relays events to Kafka.
Choreography-based Saga	Multi-service (<code>policy</code> , <code>premium</code> , <code>payment</code>)	Coordinates distributed transactions without a centralized orchestrator. Services emit domain events; downstream services react or trigger compensating transactions (<code>PolicySagaCompensationConsumer</code>).
Circuit Breaker & Retry	<code>claims-service</code> via Resilience4j	Protects synchronous OpenFeign calls to <code>policy-service</code> and <code>provider-service</code> from cascading latency or service outages with configurable failure rate thresholds and exponential backoffs.
Database-per-Service	All 13 business microservices	Enforces domain boundary autonomy. Zero shared databases or cross-service SQL joins; data exchange occurs exclusively via REST APIs or Kafka events.
API Gateway / Reverse Proxy	<code>api-gateway</code> (Spring Cloud Gateway)	Centralizes authentication, SSL termination, CORS pre-flight, and dynamic route resolution via Eureka.

17.2 Creational & Structural Patterns

Pattern	Where Used	Implementation Details & Purpose
Builder Pattern	All DTOs, Entities, and Event Envelopes	Utilizes Lombok's <code>@Builder</code> to cleanly instantiate complex immutable objects (e.g. <code>ClaimAdjudication</code> , <code>EventEnvelope</code>) with readable, parameter-safe syntax.
Factory Pattern	<code>notification-service</code> , <code>payment-service</code>	<code>NotificationFactory</code> instantiates specific email or SMS channel dispatchers based on recipient preferences. <code>PaymentProcessorFactory</code> instantiates UPI, Card, or NetBanking simulation handlers.
Adapter Pattern	<code>payment-service</code> , <code>document-service</code>	Adapts raw third-party payment gateway callbacks and local file storage paths into standardized internal domain DTOs.
Singleton Pattern	All Spring Beans	Enforced by the Spring IoC container for all <code>@Service</code> , <code>@Repository</code> , and <code>@Component</code> beans, ensuring thread-safe, single-instance lifecycle management.

17.3 Behavioral Patterns

Pattern	Where Used	Implementation Details & Purpose
Strategy Pattern	<code>quotation-service</code> , <code>claims-service</code>	<code>PremiumCalculationStrategy</code> selects different actuarial algorithms based on plan type (Individual vs Family Floater). <code>AdjudicationRuleStrategy</code> applies modular validation rules across claim stages.
Interceptor / Filter Pattern	<code>api-gateway</code> , <code>hims-ui</code>	<code>JwtAuthenticationFilter</code> intercepts HTTP requests to populate the SecurityContext; Angular <code>JwtInterceptor</code> injects bearer tokens on all client calls.
Repository Pattern	Spring Data JPA Repositories	Abstract data access behind <code>JpaRepository</code> interfaces, decoupling domain business logic from specific SQL queries and Hibernate ORM mechanics.

18. FAILURE SCENARIOS & RECOVERY STRATEGIES

To ensure 99.99% operational availability and prevent financial loss, HIMS implements automated mitigation for critical failure modes:

18.1 Failure Scenarios Matrix

#	Failure Scenario	Trigger / Root Cause	Immediate System Impact	Automated Recovery / Mitigation Strategy
1	Payment Processing Failure	Insufficient funds, gateway timeout, or bank decline during premium checkout.	Transaction marked FAILED . Policy remains in ISSUED status (not active).	System returns clear error code to user. Premium schedule maintains pending balance. User can retry with an alternate card or UPI method. No duplicate policies created.
2	Apache Kafka Broker Outage	Network partition, broker crash, or disk saturation on port 9092.	Real-time event consumption pauses; notifications and analytics delayed.	Outbox Buffer Protection: Microservices do not fail user requests. Events remain safely buffered in MySQL outbox_events with status PENDING . Once Kafka recovers, scheduled relayers resume dispatch automatically.
3	Downstream Policy Service Unavailable	policy-service pod crashes during claim submission eligibility check.	OpenFeign call from claims-service encounters socket timeout.	Resilience4j Circuit Breaker: Activates fallback method after 50% failure rate. Claim is transitioned to IN_REVIEW (Queued for offline validation) rather than throwing an HTTP 500 error to the hospital.
4	Duplicate Claim Submission Attempt	Hospital automation software submits the same hospital admission twice within seconds.	Potential risk of double claim payout.	Idempotency Barrier: The X-Idempotency-Key or composite natural key (patientId + providerCode + admissionDate) intercepts the second request, returning the existing claim reference immediately with zero duplicate database rows.
5	Expired Quotation Conversion	Customer attempts to issue a policy using a quote generated 45 days ago (validity: 30 days).	Pricing risk due to stale age/actuarial brackets.	PolicyService validates quote.getExpiryDate().isBefore(LocalDate.now()) . Rejects transaction with HTTP 400 (QUOTE_EXPIRED) and prompts the user to recalculate a fresh quote.
6	Database Connection Pool Saturation	Surge in concurrent claim submissions during regional epidemic.	HikariCP pool exhausted, queries queueing.	Fast fail configuration with HikariCP connection-timeout: 3000ms . Non-critical analytical queries routed to read-only replica databases or deferred to Kafka consumers.

19. IMPLEMENTED VS UNIMPLEMENTED FEATURES

To ensure complete academic and technical transparency during your mentor demonstration, this section explicitly delineates features that are **fully functional in source code** versus those that are **architecturally documented but simplified or planned for V2**.

19.1 Feature Implementation Matrix

Domain / Component	Feature Description	Status in Current Codebase	Verification & Implementation Notes
Core Architecture	17 Microservices Structure	✓ IMPLEMENTED	All 17 services configured in root Maven POM and <code>config-repo/</code> .
Service Discovery	Netflix Eureka Registry	✓ IMPLEMENTED	Running on port 8761 with automatic heartbeat registration.
Configuration	Centralized Spring Cloud Config	✓ IMPLEMENTED	Running on port 8888 reading from <code>config-repo/*.yaml</code> .
API Routing	Spring Cloud Gateway with CORS	✓ IMPLEMENTED	Running on port 8080 with dynamic path discovery and JWT filters.
Authentication	JWT Generation & BCrypt Hashing	✓ IMPLEMENTED	Implemented in <code>identity-service</code> with 24-hr expiry and HMAC-SHA256.
Authorization	Role-Based Access Control (RBAC)	✓ IMPLEMENTED	<code>ROLE_ADMIN</code> , <code>ROLE_CUSTOMER</code> , <code>ROLE_UNDERWRITER</code> , <code>ROLE_CLAIMS_OFFICER</code> enforced via <code>@PreAuthorize</code> .
Customer Master	KYC & Profile Management	✓ IMPLEMENTED	Customer onboarding with unique code generation, address & nominee.
Product Catalog	Products, Plans, and Benefits	✓ IMPLEMENTED	Full CRUD for individual, floater, and senior citizen plans with benefit limits.
Quotation Engine	Actuarial Premium Calculation	✓ IMPLEMENTED	Dynamic quote generation considering age, tobacco, and sum insured.
Risk Scoring	Automated Health Risk Engine	✓ IMPLEMENTED	BMI calculation, medical history scoring, and risk grade assignment.
Underwriting	Case Evaluation & Approval	✓ IMPLEMENTED	Auto-approval for low risk, underwriter queue for medium/high risk.
Policy Lifecycle	Draft, Issued, Active, Cancelled	✓ IMPLEMENTED	Policy generation, versioning, and state management.
Premium Billing	Installment Schedules	✓ IMPLEMENTED	Annual, quarterly, and monthly installment breakdown with due dates.
Payment System	Mock Gateway & Idempotency	✓ IMPLEMENTED	Simulated card/UPI payment with <code>X-Idempotency-Key</code> deduplication.
Provider Directory	In-Network Hospital Governance	✓ IMPLEMENTED	Hospital registry with network accreditation and licensing status.
Claims Engine	6-Stage Automated Adjudication	✓ IMPLEMENTED	Full SDD implementation: format, policy check, provider check, benefit rules, deductible/copay math, and EOB.

Domain / Component	Feature Description	Status in Current Codebase	Verification & Implementation Notes
Transactional Outbox	Outbox Table & Scheduled Relayer	✓ IMPLEMENTED	Implemented in <code>claims-service</code> , <code>payment-service</code> , <code>policy-service</code> .
Choreography Saga	Event Compensation on Failure	✓ IMPLEMENTED	<code>PolicySagaCompensationConsumer</code> rolls back policy state on <code>PremiumScheduleFailedEvent</code> .
Resilience4j	Circuit Breaker & Retry	✓ IMPLEMENTED	Configured on Feign clients in <code>claims-service</code> with fallback methods.
Document Storage	File Upload & Path Indexing	✓ IMPLEMENTED	Multipart file upload to local directory with metadata tracking.
Notifications	Event-Driven Notification Log	✓ IMPLEMENTED	Consumes Kafka events and logs simulated email/SMS dispatches.
Real SMTP Email	Live External Mail Server Relay	⚠ PARTIALLY IMPLEMENTED	Notification service formats emails and logs delivery to DB; real SMTP server relay disabled by default to avoid spam errors in local demo.
Real Payment PG	Live Razorpay / Stripe Gateway	⚠ PARTIALLY IMPLEMENTED	Uses high-fidelity internal gateway simulator rather than production merchant API keys.
OCR Document AI	Machine Learning Receipt OCR	✗ NOT IMPLEMENTED (V2)	Claims officer verifies uploaded invoice scans manually; automated optical character recognition planned for V2.
Biometric KYC	Government UIDAI Aadhaar OTP	✗ NOT IMPLEMENTED (V2)	Aadhaar number validated via regex format; live government biometric OTP API not integrated due to regulatory sandbox restrictions.

20. PROJECT SETUP & RUN GUIDE

Follow this exact sequence to start the Health Insurance Management System on your local development workstation.

20.1 Environment Prerequisites

- **Java:** JDK 17 (or 21 LTS) installed and configured on `JAVA_HOME`.
 - **Node.js:** v18+ (or v14+) and npm installed.
 - **MySQL Server:** MySQL 8.0 running locally on port 3306 (root password: `root` or configured in YAML).
 - **Apache Kafka & Zookeeper:** Kafka broker running on `localhost:9092` (or via Docker Compose).
 - **Maven:** 3.9+ installed and on system `PATH`.
-

20.2 Database Provisioning Script

Execute the following SQL in your MySQL client to instantiate all required microservice database schemas:

```
CREATE DATABASE IF NOT EXISTS identity_db;
CREATE DATABASE IF NOT EXISTS customer_db;
CREATE DATABASE IF NOT EXISTS product_db;
CREATE DATABASE IF NOT EXISTS quotation_db;
CREATE DATABASE IF NOT EXISTS risk_db;
CREATE DATABASE IF NOT EXISTS underwriting_db;
CREATE DATABASE IF NOT EXISTS policy_db;
CREATE DATABASE IF NOT EXISTS premium_db;
CREATE DATABASE IF NOT EXISTS payment_db;
CREATE DATABASE IF NOT EXISTS provider_db;
CREATE DATABASE IF NOT EXISTS claims_db;
CREATE DATABASE IF NOT EXISTS document_db;
CREATE DATABASE IF NOT EXISTS notification_db;
CREATE DATABASE IF NOT EXISTS reporting_db;
```

20.3 Starting Infrastructure via Docker (Optional)

If running MySQL and Kafka through Docker, start the provided docker compose file:

```
cd docker
docker-compose up -d
```

20.4 Microservices Startup Sequence

Important:

*Because microservices depend on centralized configuration and dynamic service discovery, services **MUST** be started in the following strict chronological order:*

Phase 1: Core Cloud Infrastructure (Start First)

1. Config Server (Port 8888)

```
cd services/config-server
mvn spring-boot:run
```

Wait until logs indicate: "Started ConfigServerApplication on port 8888"

2. Eureka Service Registry (Port 8761)

```
cd services/eureka-server
mvn spring-boot:run
```

Verify dashboard is accessible at: <http://localhost:8761>

3. API Gateway (Port 8080)

```
cd services/api-gateway
mvn spring-boot:run
```

Phase 2: Domain Microservices

Open separate terminal windows and run:

```
# Identity & Customer
mvn spring-boot:run -f services/identity-service/pom.xml # Port 8081
mvn spring-boot:run -f services/customer-service/pom.xml # Port 8083

# Product, Quotation, Risk & Underwriting
mvn spring-boot:run -f services/product-plan-service/pom.xml # Port 8084
mvn spring-boot:run -f services/quotation-service/pom.xml # Port 8085
mvn spring-boot:run -f services/risk-service/pom.xml # Port 8086
mvn spring-boot:run -f services/underwriting-service/pom.xml # Port 8087

# Policy, Premium, Payment & Provider
mvn spring-boot:run -f services/policy-service/pom.xml # Port 8088
mvn spring-boot:run -f services/premium-service/pom.xml # Port 8089
mvn spring-boot:run -f services/payment-service/pom.xml # Port 8090
mvn spring-boot:run -f services/provider-service/pom.xml # Port 8091

# Claims, Documents, Notifications & Reporting
mvn spring-boot:run -f services/claims-service/pom.xml # Port 8092
mvn spring-boot:run -f services/document-service/pom.xml # Port 8093
mvn spring-boot:run -f services/notification-service/pom.xml # Port 8094
mvn spring-boot:run -f services/reporting-service/pom.xml # Port 8095
```

Phase 3: Angular Frontend Application

```
cd frontend/hims-ui
npm install
npm start
```

Access the application in your browser at: <http://localhost:4200>

21. DEMO WALKTHROUGH SCRIPT

Use this proven, step-by-step 15-minute live demonstration script to present HIMS confidently to your project guide.

21.1 Live Demo Schedule & Execution Guide

Minute 00:00 – 02:00: Architecture Verification & Discovery Registry

1. **Open Browser** and navigate to Eureka Service Registry: `http://localhost:8761`
 2. **Guide Explanation:**
> "Respected Guide, as you can see on the Eureka Dashboard, all 15 business and infrastructure microservices have registered successfully. Each service runs on an isolated port with zero cross-database dependencies."
 3. Point out critical instances: `API-GATEWAY (8080)` , `POLICY-SERVICE (8088)` , `CLAIMS-SERVICE (8092)` .
-

Minute 02:00 – 05:00: Customer Onboarding, Plan Selection & Actuarial Quote

1. **Navigate to UI:** `http://localhost:4200/auth/login`
 2. Click "**Register New Customer**":
 - Name: *John Doe*, Email: *john.doe@demo.com*, Phone: *9876543210*, PAN: *ABCDE1234F*.
 - Submit form. Note the instantaneous Level 1 validation and backend generation of `CUST-2026-0001` .
 3. **Navigate to Plans:** Click on "**Health Companion Silver Plan**" (\$500,000 Sum Insured).
 4. Click "**Get Instant Quote**":
 - Add Self (Age 32), Spouse (Age 30), Non-smoker.
 - Click "**Calculate Premium**": Show the actuarial calculation: Base (\$12,000) + Age bracket factor (1.1) + Family Floater loading + 18% GST = **\$14,500.00**.
 - Note the generated Quote Number: `QU0-2026-XXXX` .
-

Minute 05:00 – 08:00: Health Risk Scoring & Underwriting Approval

1. Click "**Proceed to Health Disclosures**":
 - Input Height: 178 cm, Weight: 74 kg (BMI: 23.4 - Normal).
 - Pre-existing diseases: None declared.
 - Click "**Run Automated Risk Assessment**":
 - Show Risk Score: **18 / 100** (Risk Grade: `LOW`).

2. Underwriting Workflow:

- System displays: "Auto-underwriting qualified. Case certified for standard policy terms."
- (Optional: Log in as `ROLE_UNDERWRITER` to show the senior underwriter dashboard approving medium/high-risk cases).

3. Click "Issue Policy":

- The system calls `policy-service /api/policies/issue`.
 - Show created policy: `POL-2026-0089` with status `ISSUED`.
-

Minute 08:00 – 11:00: Saga Billing Schedule & Idempotent Premium Payment

1. Explain the Saga Event Flow to Guide:

> "Upon policy issuance, policy-service wrote a `PolicyIssuedEvent` to its transactional outbox. The scheduled relayer dispatched this to Kafka 'policy-events'. In the background, premium-service consumed this event and generated an installment schedule."

2. View Premium Schedule:

- Show the 4 quarterly installments of **\$3,625.00** each with specific due dates.

3. Pay First Installment:

- Click "Pay Installment #1".
 - Show the mock payment modal. Click "Confirm Payment".
 - Show the `X-Idempotency-Key` in the developer network tab.
 - Note: Installment transitions to `PAID`, and Policy status transitions from `ISSUED` to `ACTIVE`.
-

Minute 11:00 – 14:00: Cashless Claim Submission & 6-Stage Adjudication

1. Navigate to "Hospital Claim Portal" (`/claims/submit`):

- Policy Number: `POL-2026-0089`
- Hospital Code: `HOSP-MAX-001` (Max Healthcare - In-Network Tier 1)
- Admission Date: *Current Date - 2 days*, Discharge Date: *Current Date*
- Diagnosis ICD-10: `J18.9` (Pneumonia)
- Billed Amount: **\$45,000.00**

2. Click "Submit Claim for Adjudication":

3. Open Adjudication Inspector:

- Walk your guide through the 6 automated stages displayed on the UI:
- Stage 1: Format & Bill Completeness -> **PASSED**
- Stage 2: Policy Coverage Active Verification (OpenFeign) -> **PASSED**
- Stage 3: Hospital Network & License Accreditation (OpenFeign) -> **PASSED**
- Stage 4: ICD-10 Waiting Period Clause Evaluation -> **PASSED**
- Stage 5: Deductible (\$500) & Copay (10%) Computation:

- Billed: \$45,000.00
- Deductible Subtracted: -\$500.00
- 10% Patient Copay: -\$4,450.00
- **Net Approved Settlement: \$40,050.00**
- Stage 6: Final Decision -> **APPROVED**

4. Show generated **Explanation of Benefits (EOB)** document with printable summary.

Minute 14:00 – 15:00: Transactional Outbox Verification & Reporting Dashboard

1. Open MySQL CLI or workbench and run:

```
SELECT id, aggregate_id, event_type, status FROM claims_db.outbox_events;
```

- Show the record with `status = 'PUBLISHED'` , proving that no event is lost during outages.
 - 2. Navigate to **Executive BI Dashboard** (`/reports/dashboard`):
 - Show real-time KPI cards: Total Policies Issued, Premium Inflow, Claim Loss Ratio.
 - 3. Conclude demonstration and open the floor for mentor questions.
-

22. ADVANCED TOPICS & ARCHITECTURAL HIGHLIGHTS

22.1 Transactional Outbox Pattern Deep-Dive

One of the most complex challenges in distributed systems is the **Dual-Write Problem**:

- If an application updates its database and immediately attempts to send a message to Kafka, network failure during the Kafka call results in data written to the database without the corresponding event being published.
- If the application publishes to Kafka first and the database write fails, other services react to an event that does not exist in the primary system of record.

HIMS solves this using the **Transactional Outbox Pattern**:

1. Within a single `@Transactional` boundary in MySQL, the domain entity and an `outbox_events` row are inserted simultaneously using standard InnoDB ACID semantics.
2. An asynchronous scheduled poller (`PolicyOutboxPublisher` / `ClaimOutboxPublisher`) reads `PENDING` events from the outbox table.
3. After Kafka acknowledges receipt (`SendResult`), the poller marks the outbox status as `PUBLISHED` .
4. If Kafka is unavailable, events remain safely stored in MySQL, providing **At-Least-Once Delivery Guarantees**.

22.2 Choreography Saga vs Orchestrator Saga

HIMS utilizes **Choreography-based Sagas** rather than an Orchestrator:

- **No Single Point of Failure**: Eliminates a heavyweight central orchestrator service that could become a bottleneck.
- **Loose Coupling**: Services subscribe to events they care about without requiring an external controller to dictate their execution sequence.
- **Compensating Transactions**: Handled reactively via dedicated Kafka consumers (e.g. `PolicySagaCompensationConsumer`).

22.3 Polyglot Persistence Readiness

Because each microservice strictly encapsulates its database:

- The `claims-service` could be migrated from MySQL to MongoDB (for flexible document storage of heterogeneous hospital bills) with zero modifications to `policy-service` or `customer-service` .
 - The `reporting-service` could transition to PostgreSQL or ClickHouse for OLAP aggregation without impacting transactional systems.
-

23. CODEBASE DIRECTORY MAP

Below is the verified structural directory map of the HIMS repository:

```
health-insurance-management-system/
├── pom.xml                                # Root Maven POM managing dependencies
├── config-repo/                          # Centralized Spring Cloud Config Repository
│   ├── application.yml                  # Global shared configurations
│   ├── api-gateway.yml                 # Gateway routes, CORS, discovery locator
│   ├── identity-service.yml            # JWT secrets, token expiry, identity_db
│   ├── customer-service.yml            # customer_db, port 8083
│   ├── product-plan-service.yml        # product_db, port 8084
│   ├── quotation-service.yml           # quotation_db, port 8085
│   ├── risk-service.yml                # risk_db, port 8086
│   ├── underwriting-service.yml        # underwriting_db, port 8087
│   ├── policy-service.yml              # policy_db, Kafka policy-events, port 8088
│   ├── premium-service.yml             # premium_db, installment schedules, port 8089
│   ├── payment-service.yml             # payment_db, mock gateway, port 8090
│   ├── provider-service.yml            # provider_db, hospital network, port 8091
│   ├── claims-service.yml              # claims_db, 6-stage adjudication, Resilience4j, port 8092
│   ├── document-service.yml            # document_db, storage paths, port 8093
│   ├── notification-service.yml        # notification_db, Kafka consumers, port 8094
│   └── reporting-service.yml           # reporting_db, analytics consumers, port 8095
├── services/                            # 17 Independent Spring Boot Microservices
│   ├── config-server/                  # Spring Cloud Config Server (Port 8888)
│   ├── eureka-server/                  # Netflix Eureka Service Registry (Port 8761)
│   ├── api-gateway/                    # Spring Cloud Netty Gateway (Port 8080)
│   ├── identity-service/                # Auth, BCrypt, JWT generation (Port 8081)
│   ├── customer-service/                # Customer master, KYC (Port 8083)
│   ├── product-plan-service/            # Products, plans, benefits (Port 8084)
│   ├── quotation-service/               # Actuarial pricing engine (Port 8085)
│   ├── risk-service/                   # Health risk scoring algorithm (Port 8086)
│   ├── underwriting-service/            # Case evaluation & loadings (Port 8087)
│   ├── policy-service/                  # Policy lifecycle & Outbox (Port 8088)
│   ├── premium-service/                 # Installment schedules & Kafka listeners (Port 8089)
│   ├── payment-service/                 # Idempotent payments & mock PG (Port 8090)
│   ├── provider-service/                # Hospital network registry (Port 8091)
│   ├── claims-service/                  # 6-Stage Adjudication & Resilience4j (Port 8092)
│   ├── document-service/                # File uploads & metadata indexing (Port 8093)
│   ├── notification-service/            # Email/SMS logging consumers (Port 8094)
│   └── reporting-service/               # OLAP aggregation & KPI summaries (Port 8095)
├── frontend/                            # Angular 17 Single Page Application
│   ├── hims-ui/                        # Angular CLI Workspace
│   │   ├── package.json                # Angular dependencies
│   │   └── src/app/
│   │       ├── core/                   # Auth, Interceptors, Guards, Models
│   │       ├── features/                # Feature modules (claim, policy, quote, etc.)
│   │       └── shared/                  # Reusable UI widgets, cards, modals
├── docker/                              # Docker & Docker Compose deployment assets
│   └── docker-compose.yml              # MySQL & Kafka cluster orchestration
└── documentation/                       # Project documentation, PDF exports & guides
```

24. API REFERENCE SUMMARY TABLE

Service	Verb	Endpoint URI	Authorization / Roles	Summary Description
Identity	POST	/api/auth/register	Public	Registers a new user account with BCrypt password hashing.
Identity	POST	/api/auth/login	Public	Authenticates credentials; returns signed 24-hr JWT Bearer token.
Customer	POST	/api/customers	ROLE_CUSTOMER , ROLE_AGENT , ROLE_ADMIN	Creates master customer profile with addresses and nominees.
Customer	GET	/api/customers/{id}	Authenticated	Retrieves detailed customer profile.
Product	GET	/api/products	Public	Lists all active insurance products (Individual, Floater, Senior).
Product	GET	/api/plans	Public	Lists insurance plans with sum insured, deductible, and copay terms.
Quotation	POST	/api/quotes	Authenticated	Calculates actuarial quote based on age, tobacco, and sum insured.
Quotation	GET	/api/quotes/{id}	Authenticated	Retrieves quote summary and validity status.
Risk	POST	/api/risk-assessments	Authenticated	Calculates composite risk score (0-100) and assigns risk grade.
Underwriting	POST	/api/underwriting/evaluate	Authenticated	Evaluates quote; auto-approves low risk or queues for review.
Underwriting	PUT	/api/underwriting/{id}/approve	ROLE_UNDERWRITER , ROLE_ADMIN	Underwriter manually approves case with premium loading adjustments.
Policy	POST	/api/policies/issue	ROLE_UNDERWRITER , ROLE_AGENT , ROLE_ADMIN	Issues policy, records transactional outbox event.
Policy	GET	/api/policies/number/{num}	Authenticated	Queries policy active dates and coverage (consumed by Claims).

Service	Verb	Endpoint URI	Authorization / Roles	Summary Description
Policy	PUT	/api/policies/{id}/activate	Internal / Admin	Activates policy upon initial premium settlement.
Premium	POST	/api/premium-schedules/generate	Authenticated	Generates installment breakdown with due dates and grace periods.
Payment	POST	/api/payments/process	Authenticated (X-Idempotency-Key)	Processes mock card/UPI payment with idempotency guarantee.
Provider	GET	/api/providers	Public / Authenticated	Lists accredited healthcare providers with network status filters.
Claims	POST	/api/claims	Authenticated (X-Idempotency-Key)	Submits claim, persists dossier, initiates automated validation.
Claims	POST	/api/claims/{id}/adjudicate	ROLE_CLAIMS_OFFICER , ROLE_ADMIN	Runs 6-stage adjudication engine; calculates deductible & copay.
Claims	GET	/api/claims/{id}/eob	Authenticated	Retrieves Explanation of Benefits breakdown for patient/hospital.
Document	POST	/api/documents/upload	Authenticated	Uploads medical bills, KYC documents, or policy documents.
Reporting	GET	/api/reports/dashboard-kpis	ROLE_ADMIN	Serves aggregated business intelligence and loss ratio metrics.

25. DATABASE TABLES SUMMARY TABLE

Microservice	Database Name	Table Name	Primary Key	Purpose / Description
Identity Service	identity_db	users	id (BIGINT)	Stores user account credentials, emails, and BCrypt password hashes.
Identity Service	identity_db	roles	id (BIGINT)	Defines system roles (ROLE_ADMIN , ROLE_CUSTOMER , etc.).
Identity Service	identity_db	user_roles	(user_id , role_id)	Composite join table mapping users to their authorized roles.
Customer Service	customer_db	customers	id (BIGINT)	Master demographic profile, unique customer code, and KYC status.
Customer Service	customer_db	customer_addresses	id (BIGINT)	Permanent and communication address records for customers.
Customer Service	customer_db	customer_nominees	id (BIGINT)	Beneficiary and nominee declarations with allocation percentages.
Product Service	product_db	products	id (BIGINT)	High-level insurance product types and descriptions.
Product Service	product_db	plans	id (BIGINT)	Specific plan tiers with sum insured, base rates, and age limits.
Product Service	product_db	plan_benefits	id (BIGINT)	Specific medical coverage benefits, waiting periods, and copays.
Quotation Service	quotation_db	quotes	id (BIGINT)	Calculated quote proposals with premium totals and expiry dates.
Quotation Service	quotation_db	quote_members	id (BIGINT)	Family members covered in floater quotes with tobacco flags.
Risk Service	risk_db	risk_assessments	id (BIGINT)	Medical risk scores (0-100), risk grades, and recommended loadings.
Risk Service	risk_db	risk_factors	id (BIGINT)	Itemized risk impacts (BMI, tobacco, pre-existing conditions).
Underwriting Service	underwriting_db	underwriting_cases	id (BIGINT)	Underwriter approval records, special conditions, and loadings.
Policy Service	policy_db	policies	id (BIGINT)	System of record for policies, active coverage periods, and status.

Microservice	Database Name	Table Name	Primary Key	Purpose / Description
Policy Service	policy_db	policy_members	id (BIGINT)	Insured individuals covered under an active policy.
Policy Service	policy_db	outbox_events	id (BIGINT)	Transactional outbox table for reliable dispatch to policy-events .
Premium Service	premium_db	premium_schedules	id (BIGINT)	Billing schedule headers with total installments and balances.
Premium Service	premium_db	installments	id (BIGINT)	Individual premium installments, due dates, and paid timestamps.
Payment Service	payment_db	payments	id (BIGINT)	Payment transactions, gateway authorization codes, and methods.
Payment Service	payment_db	idempotency_records	id (BIGINT)	Unique request keys and cached responses preventing double charges.
Payment Service	payment_db	outbox_events	id (BIGINT)	Transactional outbox table for payment events.
Provider Service	provider_db	providers	id (BIGINT)	Hospital and clinic directory, network accreditation, and licensing.
Provider Service	provider_db	provider_departments	id (BIGINT)	Specialized hospital departments (Cardiology, Oncology, etc.).
Claims Service	claims_db	claims	id (BIGINT)	Core claim master record with billed and net approved amounts.
Claims Service	claims_db	claim_diagnoses	id (BIGINT)	ICD-10 diagnosis codes associated with submitted hospital bills.
Claims Service	claims_db	claim_services	id (BIGINT)	Line-item medical services (bed charges, medicines, surgeries).
Claims Service	claims_db	claim_documents	id (BIGINT)	Linkages between claim records and uploaded document files.
Claims Service	claims_db	claim_validations	id (BIGINT)	Audit of each automated stage check in the adjudication engine.
Claims Service	claims_db	claim_adjudications	id (BIGINT)	Financial adjudication calculations (deductible, copay, net amount).

Microservice	Database Name	Table Name	Primary Key	Purpose / Description
Claims Service	claims_db	explanation_of_benefits	id (BIGINT)	Formal EOB breakdown detailing insurer vs patient share.
Claims Service	claims_db	claim_payments	id (BIGINT)	Payout disbursement records sent to hospital or patient account.
Claims Service	claims_db	idempotency_records	id (BIGINT)	Unique request keys preventing duplicate claim filings.
Claims Service	claims_db	outbox_events	id (BIGINT)	Transactional outbox table for Kafka <code>claim-events</code> .
Claims Service	claims_db	audit_records	id (BIGINT)	Comprehensive audit log of entity modifications and actions.
Document Service	document_db	documents	id (BIGINT)	Metadata, physical storage paths, and MIME types of uploaded files.
Notification Service	notification_db	notifications	id (BIGINT)	Logged simulated email and SMS dispatches.
Reporting Service	reporting_db	daily_kpi_summaries	id (BIGINT)	Daily OLAP snapshots of issued policies, premium, and loss ratios.
Reporting Service	reporting_db	claim_analytics	id (BIGINT)	Adjudication processing times and provider claim distributions.

26. MENTOR VIVA / INTERVIEW QUESTIONS & ANSWERS (50+ Q&A)

This section compiles 52 deeply technical, mentor-level interview and viva questions along with comprehensive, production-grade answers to help you explain every corner of the system with total confidence.

Category A: Microservice Architecture & Distributed Systems

Q1: Why did you choose a Microservices architecture instead of a Monolith for an insurance platform?

Answer: An insurance enterprise comprises domains with vastly different operational characteristics. The Quotation and Customer browsing systems experience high, spiky read traffic, while Claims and Underwriting are computationally intensive, transactional, and audit-sensitive. A monolith would create database lock contention, single points of failure, and deployment bottlenecks. By separating the system into 17 autonomous microservices with dedicated databases, we achieve:

1. Independent horizontal scalability.
2. Fault isolation (e.g. an outage in Reporting does not stop Claim submissions).
3. Technology and schema evolution autonomy.

Q2: What is the Database-per-Service pattern, and how do you handle cross-service joins?

Answer: In HIMS, each microservice owns and connects strictly to its own dedicated MySQL schema. Cross-service database joins are strictly prohibited. When a service requires information from another domain, it obtains it through:

1. Synchronous OpenFeign REST queries (for real-time validations like checking policy active status).
2. Asynchronous event consumption via Apache Kafka (for eventual consistency, such as reporting summaries).
3. Data replication/caching of minimal immutable references (e.g. storing customerId and policyNumber directly in the claim record).

Q3: What is Netflix Eureka, and how does service discovery work in HIMS?

Answer: Eureka acts as a dynamic phonebook for microservices. Instead of hardcoding hostnames and IP addresses, each microservice registers its logical name (e.g. `policy-service`) with Eureka upon startup. The services send heartbeats every 30 seconds. Spring Cloud Gateway and OpenFeign clients query Eureka to resolve logical service names into physical IP/port pairs and perform client-side load balancing.

Q4: What is the role of Spring Cloud Config Server?

Answer: It centralizes external configuration management across all microservices using a native file repository (`config-repo/`). This allows us to modify environment parameters (such as Kafka broker addresses, database credentials, or JWT signing keys) in one place without having to recompile the service JAR files.

Q5: How does Spring Cloud Gateway route traffic? Is it blocking or non-blocking?

Answer: Spring Cloud Gateway is built on Spring 5, Project Reactor, and Netty. It is **non-blocking and reactive**, handling high volumes of concurrent requests with a small number of threads. It matches incoming request paths (e.g. `/api/policies/`), validates JWT tokens, and forwards the traffic to the resolved microservice instance via Eureka.

Category B: Distributed Transactions, Sagas & Kafka

Q6: Why can't you use standard ACID @Transactional across multiple microservices?

Answer: Standard Spring `@Transactional` relies on a single relational database connection under the control of a local transaction manager. When a transaction spans multiple microservices with separate databases, a distributed transaction requires 2-Phase Commit (2PC). However, 2PC is blocking, introduces high network latency, locks resources across network partitions, and severely degrades system availability (violating the CAP theorem).

Q7: How does HIMS implement the Saga Pattern?

Answer: HIMS implements a **Choreography-based Saga**. For example, when a policy is issued, `policy-service` emits a `PolicyIssuedEvent` to Kafka. The `premium-service` consumes this event and generates the installment schedule. If the premium schedule generation encounters an unrecoverable failure, it emits a `PremiumScheduleFailedEvent` . The `policy-service` listens for this event via `PolicySagaCompensationConsumer` and executes a **compensating transaction**, rolling back the policy status to `CANCELLED` .

Q8: What is the Dual-Write Problem, and how does the Transactional Outbox Pattern solve it?

Answer: The dual-write problem occurs when an application must update a database and publish a message to a broker like Kafka. Because a database commit and a network call to Kafka cannot be wrapped in a single atomic transaction, a crash between the two leaves the system in an inconsistent state. HIMS solves this by writing the domain entity and an `outbox_events` record into the **same local database transaction**. A scheduled background worker then reads the pending outbox records and publishes them to Kafka with at-least-once delivery guarantees.

Q9: What happens if Apache Kafka crashes while a user is issuing a policy?

Answer: Because HIMS uses the Transactional Outbox Pattern, the user's policy issuance transaction does **not** fail. The policy is successfully committed to `policy_db` , and the event is written to `outbox_events` with status `PENDING` . When Kafka recovers, the background `PolicyOutboxPublisher` resumes polling and dispatches the buffered events without any loss of data.

Q10: What is a Consumer Group in Apache Kafka?

Answer: A Consumer Group is a mechanism that allows a pool of consumer instances to divide the work of consuming and processing records from a topic. Kafka assigns each partition of a topic to exactly one consumer in the group, enabling horizontal scalability of message consumption.

Category C: Healthcare Claims & Adjudication Logic

Q11: Walk me through the 6-stage automated claim adjudication algorithm.

Answer: The 6-stage algorithm in `claims-service` consists of:

1. **Stage 1 (Completeness & Format):** Verifies non-empty bills, valid dates, and mandatory hospital diagnosis codes.
2. **Stage 2 (Policy Coverage Check):** Makes an OpenFeign call to `policy-service` to confirm the policy is in `ACTIVE` status and the admission date falls within the policy coverage window.
3. **Stage 3 (Provider Verification):** Queries `provider-service` to verify that the hospital has an active medical license and in-network accreditation.
4. **Stage 4 (Benefit & Waiting Period Check):** Evaluates the primary ICD-10 diagnosis against policy waiting period clauses (e.g. 30-day initial waiting period or 2-year pre-existing disease exclusions).
5. **Stage 5 (Financial Adjudication):** Subtracts disallowed non-medical charges, deducts the remaining annual policy deductible, and applies the plan's co-payment percentage to calculate the net approved settlement amount.
6. **Stage 6 (Authorization & EOB):** Changes status to `APPROVED`, persists the Explanation of Benefits (EOB), and writes an event to the transactional outbox.

Q12: What is the mathematical difference between a Deductible and a Co-payment?

Answer:

- **Deductible:** A fixed dollar amount that the insured must pay out-of-pocket each policy year before the insurance company pays anything (e.g., \$500).
- **Co-payment (Copay):** A fixed percentage (e.g., 10%) of the remaining allowed medical bill that the insured must pay even after the deductible has been satisfied.
- **Formula in HIMS:**

```
Net Approved = Max(0, (Billed Amount - Disallowed - Remaining Deductible) * (1 - Copay Percentage))
```

Q13: What is an Explanation of Benefits (EOB)?

Answer: An EOB is a formal accounting statement sent to the policyholder and healthcare provider detailing what medical services were billed, what amount was approved by the insurer, what amounts were disallowed, how deductibles and copays were applied, and the final net payment disbursed.

Category D: Security, JWT & RBAC

Q14: How is authentication handled across microservices?

Answer: When a user logs in via `identity-service`, credentials are verified against `identity_db` (using BCrypt). A cryptographically signed JWT token containing the username, user ID, and role claims is returned. On subsequent requests, the client passes this token in the `Authorization: Bearer <JWT>` header. Spring Cloud Gateway validates the signature at the edge, and downstream microservices parse the token using a `JwtAuthenticationFilter` to populate the Spring `SecurityContextHolder`.

Q15: Why is BCrypt used for password storage, and what is its salt factor?

Answer: BCrypt is an adaptive cryptographic hash function based on the Blowfish cipher. It incorporates a randomly generated salt to defend against rainbow table attacks and is computationally intensive to prevent brute-force hardware cracking. HIMS uses a salt work factor of 10.

Q16: How do you enforce Method-Level Security in Spring Boot?

Answer: By adding `@EnableMethodSecurity(prePostEnabled = true)` on configuration classes and decorating service or controller methods with `@PreAuthorize("hasRole('ROLE_UNDERWRITER')")`. If the user in the SecurityContext lacks the required role, Spring Security immediately throws an `AccessDeniedException` resulting in an HTTP 403 Forbidden response.

Category E: Idempotency & Fault Tolerance

Q17: How is distributed idempotency implemented in the Payment Service?

Answer: The client includes a unique UUID in the `X-Idempotency-Key` header. The `payment-service` checks its `idempotency_records` table. If the key is new, it locks the key with status `PROCESSING` and executes the payment. Upon completion, it caches the JSON response body and marks the status `COMPLETED`. If a duplicate request arrives with the same key, the service immediately returns the cached response with HTTP 200 OK without re-charging the customer's card.

Q18: How does Resilience4j Circuit Breaker work in the Claims Service?

Answer: When `claims-service` calls `policy-service` via OpenFeign, the call is monitored by a Resilience4j Circuit Breaker. Over a sliding window of 10 calls, if the failure rate exceeds 50%, the breaker transitions from **CLOSED** to **OPEN**. In the OPEN state, subsequent calls immediately fail fast to a fallback method without stressing the downstream service. After 5 seconds, the breaker enters **HALF-OPEN** to test downstream recovery with a limited number of requests.

Category F: Validation & Frontend

Q19: Explain the Three-Level Validation architecture in HIMS.

Answer:

1. **Level 1 (UI Validation):** Angular Reactive Forms with HTML5 bounds (`maxlength` , regex patterns, disabled submit buttons) provide instant user feedback.
2. **Level 2 (API/DTO Validation):** Spring Boot JSR-380 annotations (`@NotBlank` , `@Pattern` , `@Positive` , `@Size`) validate payloads at the controller boundary.
3. **Level 3 (Database Validation):** MySQL schema constraints (`NOT NULL` , `UNIQUE` , field lengths, foreign keys) ensure physical persistence integrity.

Q20: How do Angular HTTP Interceptors function in HIMS?

Answer: HIMS utilizes two interceptors:

- `JwtInterceptor` : Reads the JWT token from `TokenStorageService` and clones outgoing requests to add the `Authorization: Bearer <token>` header.
 - `ErrorInterceptor` : Catches HTTP error responses globally. If an HTTP 401 Unauthorized is detected, it logs out the user and redirects to `/auth/login` .
-

27. KNOWN DEFECTS & FUTURE ROADMAP

27.1 Technical Debt & Current System Boundaries

In accordance with professional engineering integrity, the current release contains the following defined operational boundaries:

1. **Live Third-Party SMTP Relay:** The notification service formats valid emails and logs them in `notification_db`, but external SMTP relay (e.g. Amazon SES or SendGrid) is simulated to prevent spam blacklisting during local demo testing.
2. **Live Banking Gateway Integration:** Payment processing utilizes a high-fidelity internal gateway simulator rather than live production merchant credentials (e.g. Razorpay or Stripe).
3. **Manual Invoice Data Entry:** Medical invoices and ICD-10 diagnostic codes are entered into the claim form by the user or hospital operator; automated OCR extraction from uploaded PDF scans is planned for V2.

27.2 Version 2.0 Roadmap

- **Machine Learning Document OCR:** Integrate AWS Textract or Tesseract OCR to automatically scan hospital bills and populate claim line items.
 - **Biometric Aadhaar Authentication:** Integrate live UIDAI sandbox APIs for one-time password and biometric customer KYC verification.
 - **Kubernetes & Cloud Helm Charts:** Package all 17 microservices into Helm charts for deployment on Amazon EKS or Google Kubernetes Engine (GKE).
 - **GraphQL Federation Gateway:** Implement Apollo or Spring GraphQL federation to allow mobile apps to query composite customer and policy data in a single request.
-

28. CONCLUSION & ARCHITECTURAL SUMMARY

The **Health Insurance Management System (HIMS)** represents a modern, resilient, enterprise-grade cloud microservices implementation. By strictly enforcing **Domain-Driven Design (DDD)**, **Database-per-Service isolation**, the **Transactional Outbox Pattern**, **Choreography-based Sagas**, and a **Three-Level Validation strategy**, the platform eliminates the fragility and performance bottlenecks of legacy insurance monoliths.

The system comprehensively fulfills both academic and industrial standards for distributed architectures, delivering sub-second quote estimation, automated risk scoring, reliable policy issuance, and a robust 6-stage algorithmic claim adjudication engine.
